

SOLIDWORKS API (.NET VBA C#) — Examples

(30/50): Get_Mates_Example_CSharp- Get_Number_of_Lines_Flat-pattern_Drawing

SOLIDWORKS API Documentation (converted from CHM)

Get Mates Example (C#)

This example shows how to get all of the mates (IMate2 and IMateInPlace objects) for all components in an assembly.

```
//-----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\advdrawings\bladed shaft.sldasm.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Gets the components and mates.  
// 2. Examine the Immediate window.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
namespace GetMatesComponent_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel;  
            Component2 swComponent;  
            AssemblyDoc swAssembly;  
            object[] Components = null;  
            object[] Mates = null;  
            Mate2 swMate;  
            MateInPlace swMateInPlace;  
            int numMateEntities = 0;  
            int typeOfMate = 0;  
            int i = 0;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swAssembly = (AssemblyDoc)swModel;  
            Components = (Object[])swAssembly.GetComponents(false);  
            foreach (Object SingleComponent in Components)  
            {  
                swComponent = (Component2)SingleComponent;  
                Debug.Print("Name of component: " + swComponent.Name2);  
                Mates = (Object[])swComponent.GetMates();  
                if ((Mates != null))  
                {  
                    foreach (Object SingleMate in Mates)  
                    {  
                        if (SingleMate is SolidWorks.Interop.sldworks.Mate2)  
                        {  
                            swMate = (Mate2)SingleMate;  
                            typeOfMate = swMate.Type;  
                            switch (typeOfMate)  
                            {  
                                case 0:  
                                    Debug.Print(" Mate type: Coincident");  
                                    break;  
                                case 1:  
                                    Debug.Print(" Mate type: Concentric");  
                                    break;  
                                case 2:  
                                    Debug.Print(" Mate type: Perpendicular");  
                                    break;  
                                case 3:  
                                    Debug.Print(" Mate type: Parallel");  
                                    break;  
                                case 4:  
                                    Debug.Print(" Mate type: Tangent");  
                                    break;  
                                case 5:  
                                    Debug.Print(" Mate type: Distance");  
                                    break;  
                                case 6:  
                                    Debug.Print(" Mate type: Angle");  
                                    break;  
                            }  
                        }  
                    }  
                }  
            }  
        }  
    }  
}
```

```

        case 7:
            Debug.Print(" Mate type: Unknown");
            break;
        case 8:
            Debug.Print(" Mate type: Symmetric");
            break;
        case 9:
            Debug.Print(" Mate type: CAM follower");
            break;
        case 10:
            Debug.Print(" Mate type: Gear");
            break;
        case 11:
            Debug.Print(" Mate type: Width");
            break;
        case 12:
            Debug.Print(" Mate type: Lock to sketch");
            break;
        case 13:
            Debug.Print(" Mate type: Rack pinion");
            break;
        case 14:
            Debug.Print(" Mate type: Max mates");
            break;
        case 15:
            Debug.Print(" Mate type: Path");
            break;
        case 16:
            Debug.Print(" Mate type: Lock");
            break;
        case 17:
            Debug.Print(" Mate type: Screw");
            break;
        case 18:
            Debug.Print(" Mate type: Linear coupler");
            break;
        case 19:
            Debug.Print(" Mate type: Universal joint");
            break;
        case 20:
            Debug.Print(" Mate type: Coordinate");
            break;
        case 21:
            Debug.Print(" Mate type: Slot");
            break;
        case 22:
            Debug.Print(" Mate type: Hinge");
            //
            break;
        // Add new mate types introduced after SOLIDWORKS 2010 FCS here
    }
}
if (SingleMate is SolidWorks.Interop.sldworks.MateInPlace)
{
    swMateInPlace = (MateInPlace)SingleMate;
    numMateEntities = swMateInPlace.GetMateEntityCount();
    for (i = 0; i <= numMateEntities - 1; i++)
    {
        Debug.Print(" Mate component name: " + swMateInPlace.get_MateComponentName(i));
        Debug.Print(" Type of Inplace mate entity: " + swMateInPlace.get_MateEntityType(i));
    }
}
Debug.Print("");
}
}
}
public SldWorks swApp;
}

```

Get Mates Example (VBA)

This example shows how to get all of the mates (IMate2 and IMateInPlace objects) in an assembly document.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\advdrawings\bladed shaft.sldasm.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the components and mates.  
' 2. Examine the Immediate window.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swComponent As SldWorks.Component2  
Dim swAssembly As SldWorks.AssemblyDoc  
Dim Components As Variant  
Dim SingleComponent As Variant  
Dim Mates As Variant  
Dim SingleMate As Variant  
Dim swMate As SldWorks.Mate2  
Dim swMateInPlace As SldWorks.MateInPlace  
Dim numMateEntities As Long  
Dim typeOfMate As Long  
Dim i As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swAssembly = swModel  
  
    Components = swAssembly.GetComponents(False)  
    For Each SingleComponent In Components  
        Set swComponent = SingleComponent  
        Debug.Print "Name of component: " & swComponent.Name2  
        Mates = swComponent.GetMates()  
        If (Not IsEmpty(Mates)) Then  
            For Each SingleMate In Mates  
                If TypeOf SingleMate Is SldWorks.Mate2 Then  
                    Set swMate = SingleMate  
                    typeOfMate = swMate.Type  
                    Select Case typeOfMate  
                        Case 0  
                            Debug.Print " Mate type: Coincident"  
                        Case 1  
                            Debug.Print " Mate type: Concentric"  
                        Case 2  
                            Debug.Print " Mate type: Perpendicular"  
                        Case 3  
                            Debug.Print " Mate type: Parallel"  
                        Case 4  
                            Debug.Print " Mate type: Tangent"  
                        Case 5  
                            Debug.Print " Mate type: Distance"  
                        Case 6  
                            Debug.Print " Mate type: Angle"  
                        Case 7  
                            Debug.Print " Mate type: Unknown"  
                        Case 8
```

```

        Debug.Print " Mate type: Symmetric"
    Case 9
        Debug.Print " Mate type: CAM follower"
    Case 10
        Debug.Print " Mate type: Gear"
    Case 11
        Debug.Print " Mate type: Width"
    Case 12
        Debug.Print " Mate type: Lock to sketch"
    Case 13
        Debug.Print " Mate type: Rack pinion"
    Case 14
        Debug.Print " Mate type: Max mates"
    Case 15
        Debug.Print " Mate type: Path"
    Case 16
        Debug.Print " Mate type: Lock"
    Case 17
        Debug.Print " Mate type: Screw"
    Case 18
        Debug.Print " Mate type: Linear coupler"
    Case 19
        Debug.Print " Mate type: Universal joint"
    Case 20
        Debug.Print " Mate type: Coordinate"
    Case 21
        Debug.Print " Mate type: Slot"
    Case 22
        Debug.Print " Mate type: Hinge"
    ' Add new mate types introduced after SOLIDWORKS 2010 FCS here
End Select
End If
If TypeOf SingleMate Is SldWorks.MateInPlace Then
    Set swMateInPlace = SingleMate
    numMateEntities = swMateInPlace.GetMateEntityCount
    For i = 0 To numMateEntities - 1
        Debug.Print " Mate component name: " & swMateInPlace.MateComponentName(i)
        Debug.Print " Type of Inplace mate entity: " & swMateInPlace.MateEntityType(i)
    Next i
End If
Next
End If
Debug.Print ""
Next
End Sub

```

Get Mates Example (VB.NET)

This example shows how to get all of the mates (IMate2 and IMateInPlace objects) for all components in an assembly.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\advdrawings\bladed shaft.sldasm.  
' 2. Open the Immediate window.  
'-----  
' Postconditions:  
' 1. Gets the components and mates.  
' 2. Examine the Immediate window.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
    Public Sub main()  
  
        Dim swModel As ModelDoc2  
        Dim swComponent As Component2  
        Dim swAssembly As AssemblyDoc  
        Dim Components As Object  
        Dim SingleComponent As Object  
        Dim Mates As Object  
        Dim SingleMate As Object  
        Dim swMate As Mate2  
        Dim swMateInPlace As MateInPlace  
        Dim numMateEntities As Integer  
        Dim typeOfMate As Integer  
        Dim i As Integer  
  
        swModel = swApp.ActiveDoc  
        swAssembly = swModel  
  
        Components = swAssembly.GetComponents(False)  
        For Each SingleComponent In Components  
            swComponent = SingleComponent  
            Debug.Print("Name of component: " & swComponent.Name2)  
            Mates = swComponent.GetMates()  
            If Not Mates Is Nothing Then  
                For Each SingleMate In Mates  
                    If TypeOf SingleMate Is SolidWorks.Interop.sldworks.Mate2 Then  
                        swMate = SingleMate  
                        typeOfMate = swMate.Type  
                        Select Case typeOfMate  
                            Case 0  
                                Debug.Print(" Mate type: Coincident")  
                            Case 1  
                                Debug.Print(" Mate type: Concentric")  
                            Case 2  
                                Debug.Print(" Mate type: Perpendicular")  
                            Case 3  
                                Debug.Print(" Mate type: Parallel")  
                            Case 4  
                                Debug.Print(" Mate type: Tangent")  
                            Case 5  
                                Debug.Print(" Mate type: Distance")  
                            Case 6  
                                Debug.Print(" Mate type: Angle")  
                            Case 7  
                                Debug.Print(" Mate type: Unknown")  
                            Case 8  
                                Debug.Print(" Mate type: Symmetric")  
                            Case 9  
                                Debug.Print(" Mate type: CAM follower")  
                            Case 10  
                                Debug.Print(" Mate type: Gear")  
                            Case 11  
                                Debug.Print(" Mate type: Width")  
                            Case 12
```

```

        Debug.Print(" Mate type: Lock to sketch")
    Case 13
        Debug.Print(" Mate type: Rack pinion")
    Case 14
        Debug.Print(" Mate type: Max mates")
    Case 15
        Debug.Print(" Mate type: Path")
    Case 16
        Debug.Print(" Mate type: Lock")
    Case 17
        Debug.Print(" Mate type: Screw")
    Case 18
        Debug.Print(" Mate type: Linear coupler")
    Case 19
        Debug.Print(" Mate type: Universal joint")
    Case 20
        Debug.Print(" Mate type: Coordinate")
    Case 21
        Debug.Print(" Mate type: Slot")
    Case 22
        Debug.Print(" Mate type: Hinge") '
        ' Add new mate types introduced after SOLIDWORKS 2010 FCS here
    End Select
End If
If TypeOf SingleMate Is SolidWorks.Interop.sldworks.MateInPlace Then
    swMateInPlace = SingleMate
    numMateEntities = swMateInPlace.GetMateEntityCount
    For i = 0 To numMateEntities - 1
        Debug.Print(" Mate component name: " & swMateInPlace.MateComponentName(i))
        Debug.Print(" Type of Inplace mate entity: " & swMateInPlace.MateEntityType
e(i))
    Next i
    Debug.Print("")
End If
Next
End If
Debug.Print("")
Next

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get Maximum Edge and Vertex Gaps Example (C#)

This example returns the same data returned by the **Tools > Check** dialog box when the check boxes, **Maximum edge gap** and **Maximum vertex gap**, are selected.

This example shows how to iterate over each edge and vertex in a part to obtain:

- * Whether an edge or vertex is:
 - * tolerant (distance to neighbor is greater than 5.0E-9 mm).
 - * exact (distance to neighbor is less than or equal to 5.0E-9 mm).
- * Gap size (or tolerance) in mm of each edge or vertex.
- * Maximum edge and vertex gaps (or tolerances) in mm for the part.

```
//-----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\assemblymates\head.sldprt.  
// 2. Open an Immediate window.  
//  
// Postconditions: Inspect the Immediate window for tolerance data for all edges  
// and vertexes.  
//  
// NOTE: Because the model is used elsewhere, do not save changes.  
//-----
```

```
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
namespace GetMaximumEdgeandVertexGaps_CSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            PartDoc swPart = default(PartDoc);  
            object[] vBodyArr = null;  
            Body2 swBody = default(Body2);  
            object[] vEdgeArr = null;  
            Edge swEdge = default(Edge);  
            object[] vVertexArr = null;  
            Vertex swVertex = default(Vertex);  
            FaultEntity swFaultEnt = default(FaultEntity);  
            bool bRet = false;  
            double tol = 0.0;  
            Entity swTanEnt = default(Entity);
```

```

swModel = (ModelDoc2)swApp.ActiveDoc;
swPart = (PartDoc)swModel;
Debug.Print("File = " + swModel.GetPathName());
vBodyArr = (Object[])swPart.GetBodies2((int)swBodyType_e.swAllBodies, true);
foreach ( Object vBody in vBodyArr)
{
    swBody = (Body2)vBody;
    Debug.Print(" Body[" + swBody.GetType() + "]" --> " +
swBody.GetSelectionId());
    vEdgeArr = (Object[])swBody.GetEdges();
    vVertexArr = (Object[])swBody.GetVertices();
    double maxEdgeGap = 0;
    maxEdgeGap = 0.0;
    foreach (Object vEdge in vEdgeArr)
    {
        // Find maximum edge gap
        swEdge = (Edge)vEdge;
        bRet = swEdge.IsTolerant(out tol);
        if (maxEdgeGap < tol)
        {
            maxEdgeGap = tol;
        }
        if ((!(bRet == false)))
        {
            Debug.Print("Edge is tolerant: ");
            Debug.Print(" Tolerance = " + tol);
            swTanEnt = (Entity)swEdge;
            bRet = swTanEnt.Select4(true, null);
        }
        else
        {
            Debug.Print("Edge is exact: Tolerance = " + tol);
        }
        swFaultEnt = swEdge.Check;
    }
    Debug.Print("");
    Debug.Print("Maximum edge gap is " + maxEdgeGap);
    Debug.Print("");
    double maxVertexGap = 0;
    maxVertexGap = 0.0;
    foreach (Object vVertex in vVertexArr)
    {
        // Find maximum vertex gap

```

```

        swVertex = (Vertex)vVertex;
        bRet = swVertex.IsTolerant(out tol);
        if (maxVertexGap < tol)
        {
            maxVertexGap = tol;
        }
        if (!(bRet == false))
        {
            Debug.Print("Vertex is tolerant: ");
            Debug.Print(" Tolerance = " + tol);
            swTanEnt = (Entity)swVertex;
            bRet = swTanEnt.Select4(true, null);
        }
        else
        {
            Debug.Print("Vertex is exact: Tolerance = " + tol);
        }
    }
    Debug.Print("");
    Debug.Print("Maximum vertex gap is " + maxVertexGap);
    Debug.Print("");
}

}
public SldWorks swApp;
}
}

```

Get Maximum Edge and Vertex Gaps Example (VBA)

This example returns the same data returned by the **Tools > Check** dialog box when the check boxes, **Maximum edge gap** and **Maximum vertex gap**, are selected.

This example shows how to iterate over each edge and vertex in a part to obtain:

- * Whether an edge or vertex is:
 - * tolerant (distance to neighbor is greater than 5.0E-9 mm).
 - * exact (distance to neighbor is less than or equal to 5.0E-9 mm).
- * Gap size (or tolerance) in mm of each edge or vertex.
- * Maximum edge and vertex gaps (or tolerances) in mm for the part.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\assemblymates\head.sldprt.  
' 2. Open an Immediate window.  
'  
' Postconditions: Inspect the Immediate window for tolerance data for all edges  
' and vertexes.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
' -----
```

Option Explicit

Sub main()

```
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
    Dim swPart               As SldWorks.PartDoc  
    Dim vBodyArr             As Variant  
    Dim vBody                As Variant  
    Dim swBody               As SldWorks.Body2  
    Dim vEdgeArr             As Variant  
    Dim vEdge                As Variant  
    Dim swEdge               As SldWorks.Edge  
    Dim vVertexArr           As Variant  
    Dim vVertex              As Variant  
    Dim swVertex             As SldWorks.Vertex  
    Dim swFaultEnt           As SldWorks.FaultEntity  
    Dim bRet                 As Boolean  
    Dim tol                  As Double  
    Dim swTanEnt              As SldWorks.Entity
```

```
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swPart = swModel  
    Debug.Print "File = " & swModel.GetPathName
```

```

vBodyArr = swPart.GetBodies2(swAllBodies, True)
For Each vBody In vBodyArr
    Set swBody = vBody
    Debug.Print "  Body[" & swBody.GetType & "]" --> " & swBody.GetSelectionId
    vEdgeArr = swBody.GetEdges
    vVertexArr = swBody.GetVertices

    Dim maxEdgeGap As Double
    maxEdgeGap = 0#
    For Each vEdge In vEdgeArr
        ' Find maximum edge gap
        Set swEdge = vEdge
        bRet = swEdge.IsTolerant(tol)
        If maxEdgeGap < tol Then
            maxEdgeGap = tol
        End If
        If (Not (bRet = False)) Then
            Debug.Print "Edge is tolerant: "
            Debug.Print "          Tolerance = " & tol
            Set swTanEnt = swEdge
            bRet = swTanEnt.Select4(True, Nothing)

            Else
                Debug.Print "Edge is not tolerant:  Tolerance = " & tol
            End If
        Set swFaultEnt = swEdge.Check
    Next vEdge
    Debug.Print ""
    Debug.Print "Maximum edge gap is " & maxEdgeGap
    Debug.Print ""

    Dim maxVertexGap As Double
    maxVertexGap = 0#

    For Each vVertex In vVertexArr
        ' Find maximum vertex gap
        Set swVertex = vVertex
        bRet = swVertex.IsTolerant(tol)

        If maxVertexGap < tol Then
            maxVertexGap = tol
        End If
    
```

```

    If (Not (bRet = False)) Then
        Debug.Print "Vertex is tolerant:  "
        Debug.Print "          Tolerance = " & tol
        Set swTanEnt = swVertex
        bRet = swTanEnt.Select4(True, Nothing)
    Else
        Debug.Print "Vertex is not tolerant:  Tolerance = " & tol
    End If
Next vVertex
Debug.Print ""
Debug.Print "Maximum vertex gap is " & maxVertexGap
Debug.Print ""

Next vBody
End Sub

```

Get Maximum Edge and Vertex Gaps Example (VB.NET)

This example returns the same data returned by the **Tools > Check** dialog box when the check boxes, **Maximum edge gap** and **Maximum vertex gap**, are selected.

This example shows how to iterate over each edge and vertex in a part to obtain:

- * Whether an edge or vertex is:
 - * tolerant (distance to neighbor is greater than 5.0E-9 mm).
 - * exact (distance to neighbor is less than or equal to 5.0E-9 mm).
- * Gap size (or tolerance) in mm of each edge or vertex.
- * Maximum edge and vertex gaps (or tolerances) in mm for the part.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\assemblymates\head.sldprt.  
' 2. Open an Immediate window.  
'  
' Postconditions: Inspect the Immediate window for tolerance data for all edges  
' and vertexes.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
' -----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
Partial Class SolidWorksMacro  
Sub main()  
    Dim swModel As ModelDoc2  
    Dim swPart As PartDoc  
    Dim vBodyArr As Object  
    Dim vBody As Object  
    Dim swBody As Body2  
    Dim vEdgeArr As Object  
    Dim vEdge As Object  
    Dim swEdge As Edge  
    Dim vVertexArr As Object  
    Dim vVertex As Object  
    Dim swVertex As Vertex  
    Dim swFaultEnt As FaultEntity  
    Dim bRet As Boolean  
    Dim tol As Double  
    Dim swTanEnt As Entity  
  
    swModel = swApp.ActiveDoc  
    swPart = swModel
```

```

Debug.Print("File = " & swModel.GetPathName)
vBodyArr = swPart.GetBodies2(swBodyType_e.swAllBodies, True)
For Each vBody In vBodyArr
    swBody = vBody
    Debug.Print("  Body[" & swBody.GetType & "]" --> " & swBody.GetSelectionId)
    vEdgeArr = swBody.GetEdges
    vVertexArr = swBody.GetVertices
    Dim maxEdgeGap As Double
    maxEdgeGap = 0.0#
    For Each vEdge In vEdgeArr
        ' Find maximum edge gap
        swEdge = vEdge
        bRet = swEdge.IsTolerant(tol)
        If maxEdgeGap < tol Then
            maxEdgeGap = tol
        End If
        If (Not (bRet = False)) Then
            Debug.Print("Edge is tolerant: ")
            Debug.Print("          Tolerance = " & tol)
            swTanEnt = swEdge
            bRet = swTanEnt.Select4(True, Nothing)
        Else
            Debug.Print("Edge is exact:  Tolerance = " & tol)
        End If
        swFaultEnt = swEdge.Check
    Next vEdge
    Debug.Print("")
    Debug.Print("Maximum edge gap is " & maxEdgeGap)
    Debug.Print("")
    Dim maxVertexGap As Double
    maxVertexGap = 0.0#
    For Each vVertex In vVertexArr
        ' Find maximum vertex gap
        swVertex = vVertex
        bRet = swVertex.IsTolerant(tol)
        If maxVertexGap < tol Then
            maxVertexGap = tol
        End If
        If (Not (bRet = False)) Then
            Debug.Print("Vertex is tolerant: ")
            Debug.Print("          Tolerance = " & tol)
            swTanEnt = swVertex
            bRet = swTanEnt.Select4(True, Nothing)
        End If
    Next vVertex

```

```
        Else
            Debug.Print("Vertex is exact:  Tolerance = " & tol)
        End If
    Next vVertex
    Debug.Print("")
    Debug.Print("Maximum vertex gap is " & maxVertexGap)
    Debug.Print("")
Next vBody
End Sub
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class
```

Get Messages Example (VBA)

This example shows how to get the number of messages (up to the last 20), including their text, resource IDs, and type, in the current SOLIDWORKS Design session.

```
' -----  
' Preconditions:  
' 1. Open an assembly.  
' 2. Open, edit, close, and save a part in the assembly,  
'    but do not rebuild the part.  
' 3. Close the assembly, but do not save changes  
'    made to the assembly.  
' 4. Open the Immediate window.  
'  
' Postconditions: Examine the Immediate window.  
' -----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
  
Dim vMsgs As Variant  
  
Dim vMsgIds As Variant  
  
Dim vMsgTypes As Variant  
Dim count As Long  
  
Dim i As Integer  
  
Sub main()  
  
Set swApp = Application.SldWorks  
  
count = swApp.GetErrorMessages(vMsgs, vMsgIds, vMsgTypes)  
  
Debug.Print "Number of messages = " & count  
  
For i = 0 To (count - 1)  
    Debug.Print " "  
    Debug.Print "Message = " & vMsgs(i)  
    Debug.Print "Resource ID of this message = " & vMsgIds(i)  
    Debug.Print "Type of message = " & vMsgTypes(i)  
    Debug.Print " "  
Next i  
  
End Sub
```

Get Minimum Radius of Bodies Example (C#)

This example shows how to get the minimum radius of each body in a multibody part.

```
//-----  
// Preconditions:  
// 1. Verify that the specified part document  
//    to open exists.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Opens the specified part document as read only.  
// 2. Gets the minimum radius of each body.  
// 3. Examine the Immediate window.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace Macro1CSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            PartDoc swPart = default(PartDoc);  
            object[] bodies = null;  
            Body2 abody = default(Body2);  
            double radius = 0;  
            string fileName = null;  
            int errors = 0;  
            int warnings = 0;  
            int i = 0;  
  
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\multibody\\multi_inter.sldprt";  
            swPart = (PartDoc)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOp  
tions_e.swOpenDocOptions_ReadOnly, "", ref errors, ref warnings);  
            bodies = (object[])swPart.GetBodies2((int)swBodyType_e.swAllBodies, false);  
            i = 1;  
            foreach (object vbody in bodies)  
            {  
                abody = (Body2)vbody;  
                radius = abody.MinimumRadius();  
                Debug.Print(abody.Name + "'s " + "minimum radius: " + radius);  
                i = i + 1;  
            }  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;  
    }  
}
```

Get_Minimum_Radius_of_Bodies_Example_VB (Topic)

SOLIDWORKS API Help

Get Minimum Radius of Bodies Example (VBA)

This example shows how to get the minimum radius of each body in a multibody part.

```
' -----  
' Preconditions:  
' 1. Verify that the specified part document  
'    to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified part document as read only.  
' 2. Gets the minimum radius of each body.  
' 3. Examine the Immediate window.  
' -----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swPart As SldWorks.PartDoc  
Dim bodies As Variant  
Dim vbody As Variant  
Dim radius As Double  
Dim fileName As String  
Dim errors As Long  
Dim warnings As Long  
Dim i As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\multibody\multi_inter.sldprt"  
    Set swPart = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART,  
swOpenDocOptions_e.swOpenDocOptions_ReadOnly, "", errors, warnings)  
    bodies = swPart.GetBodies2(SwConst.swBodyType_e.swAllBodies, False)  
  
    i = 1  
    For Each vbody In bodies  
        radius = vbody.MinimumRadius()  
        Debug.Print vbody.Name & "'s " & "minimum radius: " & radius  
        i = i + 1  
    Next vbody  
  
End Sub
```

Get_Minimum_Radius_of_Bodies_Example_VBNET (Topic)

SOLIDWORKS API Help

Get Minimum Radius of Bodies Example (VB.NET)

This example shows how to get the minimum radius of each body in a multibody part.

```
'-----
' Preconditions:
' 1. Verify that the specified part document
'    to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document as read only.
' 2. Gets the minimum radius of each body.
' 3. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swPart As PartDoc
        Dim bodies() As Object
        Dim vbody As Body2
        Dim radius As Double
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim i As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\multibody\multi_inter.sldprt"
        swPart = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOption
s_ReadOnly, "", errors, warnings)
        bodies = swPart.GetBodies2(swBodyType_e.swAllBodies, False)
        i = 1
        For Each vbody In bodies
            radius = vbody.MinimumRadius()
            Debug.Print(vbody.Name & "'s " & "minimum radius: " & radius)
            i = i + 1
        Next vbody

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Get_Mirror_Feature_Data_Example_VB (Topic)

SOLIDWORKS API Help

Get Mirror Pattern Feature Data Example (VBA)

This example shows how to get data for a mirror pattern feature.

```
' -----  
' Postconditions:  
' 1. Verify that the specified part document to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified part document.  
' 2. Creates Boss-Extrude2.  
' 3. Mirrors Boss-Extrude2 to create LPattern1.  
' 4. Mirrors Boss-Extrude1 and LPattern1 to create Mirror1.  
' 5. Gets the patterned features of Mirror1.  
' 6. Examine the FeatureManager design tree, the graphics area, and  
'    the Immediate window.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSketchSegment As SldWorks.SketchSegment  
Dim swSketchManager As SldWorks.SketchManager  
Dim swFeature As SldWorks.Feature  
Dim swFeatureManager As SldWorks.FeatureManager  
Dim swMirrorPatternFeatureData As SldWorks.MirrorPatternFeatureData  
Dim swEntity As SldWorks.Entity  
Dim swFace As SldWorks.Face2  
Dim swSelData As SldWorks.SelectData  
Dim status As Boolean  
Dim fileName As String  
Dim errors As Long  
Dim warnings As Long  
Dim faceArray As Variant  
Dim patternArray As Variant  
Dim i As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\beam with  
holes.sldprt"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
  
    'Create Boss-Extrude2  
    Set swModelDocExt = swModel.Extension  
    status = swModelDocExt.SelectByID2("", "FACE", 9.19322519657158E-02, -1.90749842767559E-02,  
3.99999999999999E-02, False, 0, Nothing, 0)  
    Set swSketchManager = swModel.SketchManager  
    Set swSketchSegment = swSketchManager.CreateCircle(0.092035, -0.020145, 0#, 0.093952, -0.029594, 0#)  
    swModel.ClearSelection2 True  
    status = swModelDocExt.SelectByID2("Arc1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)  
    Set swFeatureManager = swModel.FeatureManager  
    Set swFeature = swFeatureManager.FeatureExtrusion3(True, False, False, 0, 0, 0.01, 0.01, False,  
False, False, False, 1.74532925199433E-02, 1.74532925199433E-02, False, False, False, False, True, True,  
True, 0, 0, False)
```

```

'Create LPattern1
status = swModelDocExt.SelectByID2("Boss-Extrude2", "BODYFEATURE", 0, 0, 0, False, 4, Nothing, 0)
status = swModelDocExt.SelectByID2("", "EDGE", 4.41545689173921E-02, -3.98591118729428E-02,
4.02307394506352E-02, True, 1, Nothing, 0)
Set swFeature = swFeatureManager.FeatureLinearPattern4(3, 0.04, 1, 0.01, False, False, "NULL",
"NULL", False, False, False, False, False, False, True, True, False, False, 0, 0)

'Create Mirror1
status = swModelDocExt.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, True, 0, Nothing, 0)
status = swModelDocExt.SelectByID2("", "FACE", 0.232539736404746, -3.99999999999068E-02,
2.24388980993808E-02, True, 0, Nothing, 0)
swModel.ClearSelection2 True
status = swModelDocExt.SelectByID2("LPattern1", "BODYFEATURE", 0, 0, 0, False, 1, Nothing, 0)
status = swModelDocExt.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, True, 1, Nothing, 0)
status = swModelDocExt.SelectByID2("", "FACE", 0.232539736404746, -3.99999999999068E-02,
2.24388980993808E-02, True, 2, Nothing, 0)
Set swFeature = swFeatureManager.InsertMirrorFeature2(False, False, False, False,
swFeatureScope_e.swFeatureScope_AllBodies)

'Get Mirror1 properties
Set swMirrorPatternFeatureData = swFeature.GetDefinition
Debug.Print " " & swFeature.Name
Debug.Print " Geometry pattern = " &
swMirrorPatternFeatureData.GeometryPattern
Debug.Print " Number of mirrored faces = " &
swMirrorPatternFeatureData.GetMirrorFaceCount
Debug.Print " Type of mirrored plane (0 = face; 1 = plane) = " &
swMirrorPatternFeatureData.GetMirrorPlaneType
Debug.Print " Number of patterned features = " &
swMirrorPatternFeatureData.GetPatternFeatureCount

'Roll back to get to the faces and features
status = swMirrorPatternFeatureData.AccessSelections(swModel, Nothing)
swModel.ClearSelection2 True

If 1 = swMirrorPatternFeatureData.GetMirrorPlaneType Then
Set swFeature = swMirrorPatternFeatureData.Plane
'Cannot select a reference plane through Entity, so
'use Feature
Debug.Print " Plane = " & swFeature.Name
status = swFeature.Select2(True, 0)
Else
'Select face through Entity
Set swEntity = swMirrorPatternFeatureData.Plane
status = swEntity.Select4(True, swSelData)
End If

faceArray = swMirrorPatternFeatureData.MirrorFaceArray
If Not IsEmpty(faceArray) Then
swModel.ClearSelection2 True
status = False
For i = 0 To UBound(faceArray)
Set swFace = faceArray(i)
Set swEntity = swFace
status = swEntity.Select4(True, swSelData)
Next i
End If

patternArray = swMirrorPatternFeatureData.PatternFeatureArray
If Not IsEmpty(patternArray) Then
swModel.ClearSelection2 True
status = False
For i = 0 To UBound(patternArray)
Set swFeature = patternArray(i)
Debug.Print " Feature (" & i & ") = " &
swFeature.Name
status = swFeature.Select2(True, 0)
Next i
End If

'Cancel changes
swMirrorPatternFeatureData.ReleaseSelectionAccess

```

End Sub

Get_Mirror_Part_Information_Example_CSharp (Topic)

SOLIDWORKS API Help

Get Mirror Part Information Example (C#)

This example shows how to get information about a mirror part.

```
//-----  
// Preconditions:  
// 1. Open a part that contains a reference plane about which to mirror the part.  
// 2. Select the reference plane in the FeatureManager design tree.  
//  
// Postconditions:  
// 1. Creates and mirrors a part about the selected reference plane.  
// 2. Shows the reference plane in the graphics area.  
// 3. Inspect the Immediate window for mirror part feature data.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace GetMirrorPartFeatureData_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
  
        ModelDoc2 swModel;  
        SelectionMgr selMgr;  
        PartDoc swPart;  
        Feature mirrorFeature;  
        Feature myFeature;  
        ModelDoc2 resultPart;  
        MirrorPartFeatureData mirrorFeatData;  
        RefPlane refPlane;  
        Face2 facePlane;  
        int mirrorOptions;  
        int faceType;  
        swSelectType_e selType;  
  
        string message;  
  
        public void Main()  
        {  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
  
            if (swModel == null)  
                return;  
            if (swModel.GetType() != (int)swDocumentTypes_e.swDocPART)  
                return;  
  
            selMgr = (SelectionMgr)swModel.SelectionManager;  
  
            if (selMgr.GetSelectedObjectCount2(-1) == 0)  
                return;  
  
            selType = (swSelectType_e)selMgr.GetSelectedObjectType3(1, -1);  
            if (!(selType == swSelectType_e.swSelDATUMPLANES | selType ==  
swSelectType_e.swSelFACES))  
                return;  
        }  
    }  
}
```

```

swPart = (PartDoc)swModel;

mirrorOptions = (int)swMirrorPartOptions_e.swMirrorPartOptions_ImportSolids +
(int)swMirrorPartOptions_e.swMirrorPartOptions_ImportCustomProperties +
(int)swMirrorPartOptions_e.swMirrorPartOptions_ImportCutListProperties;

mirrorFeature = swPart.MirrorPart2(false, mirrorOptions, out resultPart);

if (mirrorFeature == null)
{
    message = "No Feature";
}
else
{
    message = "Feature: " + mirrorFeature.Name;
}

if (resultPart == null)
{
    Debug.Print("No new part. " + message);
}
else
{
    Debug.Print(resultPart.GetTitle() + ". " + message);
}

swModel = (ModelDoc2)swApp.ActiveDoc;
selMgr = (SelectionMgr)swModel.SelectionManager;
mirrorFeature.Select2(false, -1);

myFeature = (Feature)selMgr.GetSelectedObject6(1, -1);
mirrorFeatData = (MirrorPartFeatureData)myFeature.GetDefinition();

mirrorFeatData.AccessSelections(swModel, null);

Debug.Print(" Path name = " + mirrorFeatData.PathName);
Debug.Print(" Import surface bodies? = " + mirrorFeatData.SurfaceBodies);
Debug.Print(" Import solid bodies? = " + mirrorFeatData.SolidBodies);
Debug.Print(" Import custom properties? = " +
mirrorFeatData.CustomProperties);
Debug.Print(" Import cut-list properties? = " +
mirrorFeatData.CutListProperties);

faceType = mirrorFeatData.GetMirrorPlaneType();
faceType);
Debug.Print(" Mirror plane type as defined in swMirrorPlaneType_e = " +

Entity plane = default(Entity);
if (faceType == 1)
{
    refPlane = (RefPlane)mirrorFeatData.GetMirrorPlane();
    mirrorFeatData.ReleaseSelectionAccess();
    plane = (Entity)refPlane;
    plane.Select(false);
}
else
{
    facePlane = (Face2)mirrorFeatData.GetMirrorPlane();
    mirrorFeatData.ReleaseSelectionAccess();
    plane = (Entity)facePlane;
    plane.Select(false);
}

}

public SldWorks swApp;

```

}

}

Get Mirror Part Information Example (VBA)

This example shows how to get information about a mirror part.

```
' -----  
' Preconditions:  
' 1. Open a part that contains a reference plane about which to mirror the part.  
' 2. Select the reference plane in the FeatureManager design tree.  
'  
' Postconditions:  
' 1. Creates and mirrors a part about the selected reference plane.  
' 2. Shows the reference plane in the graphics area.  
' 3. Inspect the Immediate window for mirror part feature data.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim selMgr As SldWorks.SelectionMgr  
Dim swPart As SldWorks.PartDoc  
Dim mirrorFeature As SldWorks.Feature  
Dim myFeature As SldWorks.Feature  
Dim resultPart As SldWorks.ModelDoc2  
Dim mirrorFeatData As SldWorks.MirrorPartFeatureData  
Dim refPlane As SldWorks.refPlane  
Dim facePlane As SldWorks.Face2  
Dim mirrorOptions As Long  
Dim faceType As Long  
Dim selType As swSelectType_e  
Dim message As String  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    If swModel Is Nothing Then Exit Sub  
    If swModel.GetType <> swDocPART Then Exit Sub  
  
    Set selMgr = swModel.SelectionManager  
  
    If selMgr.GetSelectedObjectCount2(-1) = 0 Then Exit Sub  
  
    selType = selMgr.GetSelectedObjectType3(1, -1)  
    If Not (selType = swSelDATUMPLANES Or selType = swSelFACES) Then Exit Sub  
  
    Set swPart = swModel  
  
    mirrorOptions = swMirrorPartOptions_ImportSolids + _  
                   swMirrorPartOptions_ImportCustomProperties + _  
                   swMirrorPartOptions_ImportCutListProperties  
  
    Set mirrorFeature = swPart.MirrorPart2(False, mirrorOptions, resultPart)  
  
    If mirrorFeature Is Nothing Then  
        message = "No Feature"
```

```
Else
    message = "Feature: " & mirrorFeature.Name
End If
```

```
If resultPart Is Nothing Then
    Debug.Print "No new part. " & message
Else
    Debug.Print resultPart.GetTitle & ". " & message
End If
```

```
Set swModel = swApp.ActiveDoc
Set selMgr = swModel.SelectionManager
mirrorFeature.Select2 False, -1
```

```
Set myFeature = selMgr.GetSelectedObject6(1, -1)
Set mirrorFeatData = myFeature.GetDefinition
```

```
mirrorFeatData.AccessSelections swModel, Nothing
```

```
Debug.Print " Path name = " & mirrorFeatData.PathName
Debug.Print " Import surface bodies? = " & mirrorFeatData.SurfaceBodies
Debug.Print " Import solid bodies? = " & mirrorFeatData.SolidBodies
Debug.Print " Import custom properties? = " & mirrorFeatData.CustomProperties
Debug.Print " Import cut-list properties? = " & mirrorFeatData.CutListProperties
```

```
faceType = mirrorFeatData.GetMirrorPlaneType
Debug.Print " Mirror plane type as defined in swMirrorPlaneType_e = " & faceType
```

```
Dim plane As SldWorks.Entity
If faceType = 1 Then
    Set refPlane = mirrorFeatData.GetMirrorPlane
    mirrorFeatData.ReleaseSelectionAccess
    Set plane = refPlane
    plane.Select False
Else
    Set facePlane = mirrorFeatData.GetMirrorPlane
    mirrorFeatData.ReleaseSelectionAccess
    Set plane = facePlane
    plane.Select False
End If
```

```
End Sub
```

Get Mirror Part Information Example (VB.NET)

This example shows how to get information about a mirror part.

```
' -----  
' Preconditions:  
' 1. Open a part that contains a reference plane about which to mirror the part.  
' 2. Select the reference plane in the FeatureManager design tree.  
'  
' Postconditions:  
' 1. Creates and mirrors a part about the selected reference plane.  
' 2. Shows the reference plane in the graphics area.  
' 3. Inspect the Immediate window for mirror part feature data.  
' -----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

```
    Dim swModel As ModelDoc2  
    Dim selMgr As SelectionMgr  
    Dim swPart As PartDoc  
    Dim mirrorFeature As Feature  
    Dim myFeature As Feature  
    Dim resultPart As ModelDoc2  
    Dim mirrorFeatData As MirrorPartFeatureData  
    Dim refPlane As RefPlane  
    Dim facePlane As Face2  
    Dim mirrorOptions As Integer  
    Dim faceType As Integer  
    Dim selType As swSelectType_e  
    Dim message As String  
  
    Sub main()  
  
        swModel = swApp.ActiveDoc  
  
        If swModel Is Nothing Then Exit Sub  
        If swModel.GetType <> swDocumentTypes_e.swDocPART Then Exit Sub  
  
        selMgr = swModel.SelectionManager  
  
        If selMgr.GetSelectedObjectCount2(-1) = 0 Then Exit Sub  
  
        selType = selMgr.GetSelectedObjectType3(1, -1)  
        If Not (selType = swSelectType_e.swSelDATUMPLANES Or selType =  
swSelectType_e.swSelFACES) Then Exit Sub  
  
        swPart = swModel  
  
        mirrorOptions = swMirrorPartOptions_e.swMirrorPartOptions_ImportSolids + _  
                        swMirrorPartOptions_e.swMirrorPartOptions_ImportCustomProperties +  
--  
                        swMirrorPartOptions_e.swMirrorPartOptions_ImportCutListProperties
```

```

mirrorFeature = swPart.MirrorPart2(False, mirrorOptions, resultPart)

If mirrorFeature Is Nothing Then
    message = "No Feature"
Else
    message = "Feature: " & mirrorFeature.Name
End If

If resultPart Is Nothing Then
    Debug.Print("No new part. " & message)
Else
    Debug.Print(resultPart.GetTitle & ". " & message)
End If

swModel = swApp.ActiveDoc
selMgr = swModel.SelectionManager
mirrorFeature.Select2(False, -1)

myFeature = selMgr.GetSelectedObject6(1, -1)
mirrorFeatData = myFeature.GetDefinition

mirrorFeatData.AccessSelections(swModel, Nothing)

Debug.Print(" Path name = " & mirrorFeatData.PathName)
Debug.Print(" Import surface bodies? = " & mirrorFeatData.SurfaceBodies)
Debug.Print(" Import solid bodies? = " & mirrorFeatData.SolidBodies)
Debug.Print(" Import custom properties? = " & mirrorFeatData.CustomProperties)
Debug.Print(" Import cut-list properties? = " & mirrorFeatData.CutListProperties)

faceType = mirrorFeatData.GetMirrorPlaneType
Debug.Print(" Mirror plane type as defined in swMirrorPlaneType_e = " & faceType)

Dim plane As Entity
If faceType = 1 Then
    refPlane = mirrorFeatData.GetMirrorPlane
    mirrorFeatData.ReleaseSelectionAccess()
    plane = refPlane
    plane.Select(False)
Else
    facePlane = mirrorFeatData.GetMirrorPlane
    mirrorFeatData.ReleaseSelectionAccess()
    plane = facePlane
    plane.Select(False)
End If

End Sub

Public swApp As SldWorks

End Class

```

This example shows how to get data for a mirror pattern feature.

```
//-----
// Postconditions:
// 1. Verify that the specified part document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Creates Boss-Extrude2.
// 3. Mirrors Boss-Extrude2 to create LPattern1.
// 4. Mirrors Boss-Extrude1 and LPattern1 to create Mirror1.
// 5. Gets the patterned features of Mirror1.
// 6. Examine the FeatureManager design tree, the graphics area, and
//    the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace MirrorPatternFeatureDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchSegment swSketchSegment = default(SketchSegment);
            SketchManager swSketchManager = default(SketchManager);
            Feature swFeature = default(Feature);
            FeatureManager swFeatureManager = default(FeatureManager);
            MirrorPatternFeatureData swMirrorPatternFeatureData = default(MirrorPatternFeatureData);
            Entity swEntity = default(Entity);
            Face2 swFace = default(Face2);
            SelectData swSelData = null;
            bool status = false;
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            object[] faceArray = null;
            object[] patternArray = null;
            int i = 0;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\beam with holes.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDo
cOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            //Create Boss-Extrude2
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID("", "FACE", 0.0919322519657158, -0.0190749842767559, 0.039
999999999068, false, 0, null, 0);
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchSegment = (SketchSegment)swSketchManager.CreateCircle(0.092035, -0.020145, 0.0, 0.093
952, -0.029594, 0.0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID("Arc1", "SKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swFeature = (Feature)swFeatureManager.FeatureExtrusion3(true, false, false, 0, 0, 0.01, 0.01,
false, false, false,
false, 0.0174532925199433, 0.0174532925199433, false, false, false, false, true, true, true,
0, 0, false);

            //Create LPattern1
            status = swModelDocExt.SelectByID("Boss-
```

```

Extrude2", "BODYFEATURE", 0, 0, 0, false, 4, null, 0);
status = swModelDocExt.SelectByID2("", "EDGE", 0.0441545689173921, -0.0398591118729428, 0.040
2307394506352, true, 1, null, 0);
swFeature = (Feature)swFeatureManager.FeatureLinearPattern4(3, 0.04, 1, 0.01, false, false, "
NULL", "NULL", false, false,
false, false, false, false, true, true, false, false, 0, 0);

//Create Mirror1
status = swModelDocExt.SelectByID2("Boss-
Extrude1", "BODYFEATURE", 0, 0, 0, true, 0, null, 0);
status = swModelDocExt.SelectByID2("", "FACE", 0.232539736404746, -0.03999999999999068, 0.0224
388980993808, true, 0, null, 0);
swModel.ClearSelection2(true);
status = swModelDocExt.SelectByID2("LPattern1", "BODYFEATURE", 0, 0, 0, false, 1, null, 0);
status = swModelDocExt.SelectByID2("Boss-
Extrude1", "BODYFEATURE", 0, 0, 0, true, 1, null, 0);
status = swModelDocExt.SelectByID2("", "FACE", 0.232539736404746, -0.03999999999999068, 0.0224
388980993808, true, 2, null, 0);
swFeature = (Feature)swFeatureManager.InsertMirrorFeature2(false, false, false, false, (int)s
wFeatureScope_e.swFeatureScope_AllBodies);

//Get Mirror1 properties
swMirrorPatternFeatureData = (MirrorPatternFeatureData)swFeature.GetDefinition();
Debug.Print(" " + swFeature.Name);
Debug.Print(" Geometry pattern = " + swMirrorPatternFe
atureData.GeometryPattern);
Debug.Print(" Number of mirrored faces = " + swMirrorPatternFe
atureData.GetMirrorFaceCount());
Debug.Print(" Type of mirrored plane (0 = face; 1 = plane) = " + swMirrorPatternFe
atureData.GetMirrorPlaneType());
Debug.Print(" Number of patterned features = " + swMirrorPatternFe
atureData.GetPatternFeatureCount());

//Roll back to get to the faces and features
status = swMirrorPatternFeatureData.AccessSelections(swModel, null);
swModel.ClearSelection2(true);

if (1 == swMirrorPatternFeatureData.GetMirrorPlaneType())
{
swFeature = (Feature)swMirrorPatternFeatureData.Plane;
//Cannot select a reference plane through Entity, so
//use Feature
Debug.Print(" Plane = " + swFeature.Name);
status = swFeature.Select2(true, 0);
}
else
{
//Select face through Entity
swEntity = (Entity)swMirrorPatternFeatureData.Plane;
status = swEntity.Select4(true, swSelData);
}

faceArray = (object[])swMirrorPatternFeatureData.MirrorFaceArray;
if ((faceArray != null))
{
swModel.ClearSelection2(true);
status = false;
for (i = 0; i < faceArray.Length; i++)
{
swFace = (Face2)faceArray[i];
swEntity = (Entity)swFace;
status = swEntity.Select4(true, swSelData);
}
}

patternArray = (object[])swMirrorPatternFeatureData.PatternFeatureArray;
if ((patternArray != null))
{
swModel.ClearSelection2(true);
status = false;
for (i = 0; i < patternArray.Length; i++)
{
swFeature = (Feature)patternArray[i];
Debug.Print(" Feature (" + i + ") = " + s
wFeature.Name);
status = swFeature.Select2(true, 0);
}
}
//Cancel changes
swMirrorPatternFeatureData.ReleaseSelectionAccess();
}

```

```
    /// <summary>
    ///   The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Get Mirror Pattern Feature Data Example (VB.NET)

This example shows how to get data for a mirror pattern feature.

```
'-----  
' Postconditions:  
' 1. Verify that the specified part document to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified part document.  
' 2. Creates Boss-Extrude2.  
' 3. Mirrors Boss-Extrude2 to create LPattern1.  
' 4. Mirrors Boss-Extrude1 and LPattern1 to create Mirror1.  
' 5. Gets the patterned features of Mirror1.  
' 6. Examine the FeatureManager design tree, the graphics area, and  
'    the Immediate window.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Public Sub main()  
  
        Dim swModel As ModelDoc2  
        Dim swModelDocExt As ModelDocExtension  
        Dim swSketchSegment As SketchSegment  
        Dim swSketchManager As SketchManager  
        Dim swFeature As Feature  
        Dim swFeatureManager As FeatureManager  
        Dim swMirrorPatternFeatureData As MirrorPatternFeatureData  
        Dim swEntity As Entity  
        Dim swFace As Face2  
        Dim swSelData As SelectData = Nothing  
        Dim status As Boolean  
        Dim fileName As String  
        Dim errors As Integer  
        Dim warnings As Integer  
        Dim faceArray() As Object  
        Dim patternArray() As Object  
        Dim i As Integer  
  
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\api\beam with holes.sldprt"  
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptio  
ns_Silent, "", errors, warnings)  
  
        'Create Boss-Extrude2  
        swModelDocExt = swModel.Extension  
        status = swModelDocExt.SelectByID("", "FACE", 0.0919322519657158, -0.0190749842767559, 0.0399999  
9999999068, False, 0, Nothing, 0)  
        swSketchManager = swModel.SketchManager  
        swSketchSegment = swSketchManager.CreateCircle(0.092035, -0.020145, 0.0#, 0.093952, -0.029594, 0.  
0#)  
        swModel.ClearSelection2(True)  
        status = swModelDocExt.SelectByID("Arc1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)  
        swFeatureManager = swModel.FeatureManager  
        swFeature = swFeatureManager.FeatureExtrusion3(True, False, False, 0, 0, 0.01, 0.01, False, False  
, False, False, 0.0174532925199433, 0.0174532925199433, False, False, False, False, True, True, True, 0,  
0, False)  
  
        'Create LPattern1  
        status = swModelDocExt.SelectByID("Boss-Extrude2", "BODYFEATURE", 0, 0, 0, False, 4, Nothing, 0)  
        status = swModelDocExt.SelectByID("", "EDGE", 0.0441545689173921, -0.0398591118729428, 0.0402307  
394506352, True, 1, Nothing, 0)  
        swFeature = swFeatureManager.FeatureLinearPattern4(3, 0.04, 1, 0.01, False, False, "NULL", "NULL"  
, False, False, False, False, False, False, True, True, False, False, 0, 0)
```

```

'Create Mirror1
status = swModelDocExt.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, True, 0, Nothing, 0)
status = swModelDocExt.SelectByID2("", "FACE", 0.232539736404746, -0.0399999999999068, 0.02243889
80993808, True, 0, Nothing, 0)
swModel.ClearSelection2(True)
status = swModelDocExt.SelectByID2("LPattern1", "BODYFEATURE", 0, 0, 0, False, 1, Nothing, 0)
status = swModelDocExt.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, True, 1, Nothing, 0)
status = swModelDocExt.SelectByID2("", "FACE", 0.232539736404746, -0.0399999999999068, 0.02243889
80993808, True, 2, Nothing, 0)
swFeature = swFeatureManager.InsertMirrorFeature2(False, False, False, False, swFeatureScope_e.sw
FeatureScope_AllBodies)

'Get Mirror1 properties
swMirrorPatternFeatureData = swFeature.GetDefinition
Debug.Print(" " & swFeature.Name)
Debug.Print("      Geometry pattern" = " & swMirrorPatternFeatur
eData.GeometryPattern)
Debug.Print("      Number of mirrored faces" = " & swMirrorPatternFeatur
eData.GetMirrorFaceCount)
Debug.Print("      Type of mirrored plane (0 = face; 1 = plane)" = " & swMirrorPatternFeatur
eData.GetMirrorPlaneType)
Debug.Print("      Number of patterned features" = " & swMirrorPatternFeatur
eData.GetPatternFeatureCount)

'Roll back to get to the faces and features
status = swMirrorPatternFeatureData.AccessSelections(swModel, Nothing)
swModel.ClearSelection2(True)

If 1 = swMirrorPatternFeatureData.GetMirrorPlaneType Then
    swFeature = swMirrorPatternFeatureData.Plane
    'Cannot select a reference plane through Entity, so
    'use Feature
    Debug.Print("      Plane" = " & swFeature.Name)
    status = swFeature.Select2(True, 0)
Else
    'Select face through Entity
    swEntity = swMirrorPatternFeatureData.Plane
    status = swEntity.Select4(True, swSelData)
End If

faceArray = swMirrorPatternFeatureData.MirrorFaceArray
If Not faceArray Is Nothing Then
    swModel.ClearSelection2(True)
    status = False
    For i = 0 To UBound(faceArray)
        swFace = faceArray(i)
        swEntity = swFace
        status = swEntity.Select4(True, swSelData)
    Next i
End If

patternArray = swMirrorPatternFeatureData.PatternFeatureArray
If Not patternArray Is Nothing Then
    swModel.ClearSelection2(True)
    status = False
    For i = 0 To UBound(patternArray)
        swFeature = patternArray(i)
        Debug.Print("      Feature (" & i & ")" = " & swFea
ture.Name)
        status = swFeature.Select2(True, 0)
    Next i
End If
'Cancel changes
swMirrorPatternFeatureData.ReleaseSelectionAccess()

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get Mirror Solid Feature Data Example (VBA)

This example shows how to get data for a mirror solid feature.

```
' -----  
' Preconditions:  
' 1. Verify that the specified part document to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified part document.  
' 2. Selects a plane and solid body.  
' 3. Mirrors the solid body.  
' 4. Gets the mirror solid feature and some of its data.  
' 5. Prints to the Immediate window some mirror solid feature data.  
' 6. Examine the Immediate window, FeatureManager design tree, and graphics  
'    area.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swFeatureManager As SldWorks.FeatureManager  
Dim swFeature As SldWorks.Feature  
Dim swMirrorSolidFeatureData As SldWorks.MirrorSolidFeatureData  
Dim swBody As SldWorks.Body2  
Dim swSelectionMgr As SldWorks.SelectionMgr  
Dim swSelData As SldWorks.SelectData  
Dim status As Boolean  
Dim errors As Long  
Dim warnings As Long  
Dim fileName As String  
Dim i As Long  
Dim bodies As Variant  
  
Sub main()  
    Set swApp = Application.SldWorks  
  
    'Open part  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\multibody\multi_inter.sldprt"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
  
    'Select plane and solid body  
    Set swModelDocExt = swModel.Extension  
    status = swModelDocExt.SelectByID2("Top", "PLANE", 0, 0, 0, True, 0, Nothing, 0)  
    status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)  
    swModel.ClearSelection2 True  
    status = swModelDocExt.SelectByID2("Top", "PLANE", 0, 0, 0, False, 2, Nothing, 0)  
    status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, True, 256, Nothing, 0)  
  
    'Insert mirror solid feature  
    Set swFeatureManager = swModel.FeatureManager  
    Set swFeature = swFeatureManager.InsertMirrorFeature2(True, False, False, False,  
swFeatureScope_e.swFeatureScope_AllBodies)  
  
    'Get mirror solid feature and some of its data  
    Set swMirrorSolidFeatureData = swFeature.GetDefinition  
    Debug.Print " " & swFeature.Name
```

```
Debug.Print "    Number of bodies                = " & swMirrorSolidFeatureData.GetPatternBodyCount  
Debug.Print "    Merged bodies                  = " & swMirrorSolidFeatureData.Merge  
Debug.Print "    Knit surfaces                    = " & swMirrorSolidFeatureData.KnitSurface
```

```
'Roll back to get to the bodies  
status = swMirrorSolidFeatureData.AccessSelections(swModel, Nothing)
```

```
Set swSelectionMgr = swModel.SelectionManager  
Set swSelData = swSelectionMgr.CreateSelectData
```

```
bodies = swMirrorSolidFeatureData.PatternBodyArray  
For i = 0 To UBound(bodies)  
    Set swBody = bodies(i)  
    status = swBody.Select(True, 0)  
    Debug.Print "    Body " & i + 1 & "'s type (solid body = 0) = " & swBody.GetType  
Next i
```

```
'Release selection access  
swMirrorSolidFeatureData.ReleaseSelectionAccess
```

```
End Sub
```

Get Mirror Solid Feature Data Example (C#)

This example shows how to get data for a mirror solid feature.

```
//-----  
// Preconditions:  
// 1. Verify that the specified part document to open exists.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Opens the specified part document.  
// 2. Selects a plane and solid body.  
// 3. Mirrors the solid body.  
// 4. Gets the mirror solid feature and some of its data.  
// 5. Prints to the Immediate window some mirror solid feature data.  
// 6. Examine the Immediate window, FeatureManager design tree, and graphics  
//    area.  
//  
// NOTE: Because the part is used elsewhere, do not save changes.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace MirrorSolidFeatureDataCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            ModelDocExtension swModelDocExt = default(ModelDocExtension);  
            FeatureManager swFeatureManager = default(FeatureManager);  
            Feature swFeature = default(Feature);  
            MirrorSolidFeatureData swMirrorSolidFeatureData = default(MirrorSolidFeatureData);  
            Body2 swBody = default(Body2);  
            SelectionMgr swSelectionMgr = default(SelectionMgr);  
            SelectData swSelData = default(SelectData);  
            bool status = false;  
            int errors = 0;  
            int warnings = 0;  
            string fileName = null;  
            int i = 0;  
            object[] bodies = null;  
  
            //Open part  
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\multibody\\multi_inter.sldprt";  
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDo  
cOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);  
  
            //Select plane and solid body  
            swModelDocExt = (ModelDocExtension)swModel.Extension;  
            status = swModelDocExt.SelectByID2("Top", "PLANE", 0, 0, 0, true, 0, null, 0);  
            status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, true, 0, null, 0);  
            swModel.ClearSelection2(true);  
            status = swModelDocExt.SelectByID2("Top", "PLANE", 0, 0, 0, false, 2, null, 0);  
            status = swModelDocExt.SelectByID2("Extrude-  
Thin1", "SOLIDBODY", 0, 0, 0, true, 256, null, 0);  
  
            //Insert mirror solid feature  
            swFeatureManager = (FeatureManager)swModel.FeatureManager;  
            swFeature = (Feature)swFeatureManager.InsertMirrorFeature2(true, false, false, false, (int)sw
```

```

FeatureScope_e.swFeatureScope_AllBodies);

    //Get mirror solid feature and some of its data
    swMirrorSolidFeatureData = (MirrorSolidFeatureData)swFeature.GetDefinition();
    Debug.Print(" " + swFeature.Name);
    Debug.Print("    Number of bodies"           = " + swMirrorSolidFeatureData.GetPatternBod

yCount());
    Debug.Print("    Merged bodies"           = " + swMirrorSolidFeatureData.Merge);
    Debug.Print("    Knit surfaces"           = " + swMirrorSolidFeatureData.KnitSurface);

    //Roll back to get to the bodies
    status = swMirrorSolidFeatureData.AccessSelections(swModel, null);
    swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
    swSelData = (SelectData)swSelectionMgr.CreateSelectData();
    bodies = (object[])swMirrorSolidFeatureData.PatternBodyArray;
    for (i = 0; i < bodies.Length; i++)
    {
        swBody = (Body2)bodies[i];
        status = swBody.Select(true, 0);
        Debug.Print("    Body " + i + 1 + "'s type (solid body = 0) = " + swBody.GetType());
    }

    //Release selection access
    swMirrorSolidFeatureData.ReleaseSelectionAccess();

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Get_Mirror_Solid_Feature_Data_Example_VBNET (Topic)

SOLIDWORKS API Help

Get Mirror Solid Feature Data Example (VB.NET)

This example shows how to get data for a mirror solid feature.

```
'-----  
' Preconditions:  
' 1. Verify that the specified part document to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified part document.  
' 2. Selects a plane and solid body.  
' 3. Mirrors the solid body.  
' 4. Gets the mirror solid feature and some of its data.  
' 5. Prints to the Immediate window some mirror solid feature data.  
' 6. Examine the Immediate window, FeatureManager design tree, and graphics  
'    area.  
'  
' NOTE: Because the part is used elsewhere, do not save changes.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
    Public Sub main()  
  
        Dim swModel As ModelDoc2  
        Dim swModelDocExt As ModelDocExtension  
        Dim swFeatureManager As FeatureManager  
        Dim swFeature As Feature  
        Dim swMirrorSolidFeatureData As MirrorSolidFeatureData  
        Dim swBody As Body2  
        Dim swSelectionMgr As SelectionMgr  
        Dim swSelData As SelectData  
        Dim status As Boolean  
        Dim errors As Integer  
        Dim warnings As Integer  
        Dim fileName As String  
        Dim i As Integer  
        Dim bodies() As Object  
  
        'Open part  
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\multibody\multi_inter.sldprt"  
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptio  
ns_Silent, "", errors, warnings)  
  
        'Select plane and solid body  
        swModelDocExt = swModel.Extension  
        status = swModelDocExt.SelectByID2("Top", "PLANE", 0, 0, 0, True, 0, Nothing, 0)  
        status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)  
        swModel.ClearSelection2(True)  
        status = swModelDocExt.SelectByID2("Top", "PLANE", 0, 0, 0, False, 2, Nothing, 0)  
        status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, True, 256, Nothing, 0)  
  
        'Insert mirror solid feature  
        swFeatureManager = swModel.FeatureManager  
        swFeature = swFeatureManager.InsertMirrorFeature2(True, False, False, False, swFeatureScope_e.swF  
eatureScope_AllBodies)  
  
        'Get mirror solid feature and some of its data  
        swMirrorSolidFeatureData = swFeature.GetDefinition  
        Debug.Print(" " & swFeature.Name)  
        Debug.Print("    Number of bodies" & " " & swMirrorSolidFeatureData.GetPatternBodyCoun  
t)  
        Debug.Print("    Merged bodies" & " " & swMirrorSolidFeatureData.Merge)  
        Debug.Print("    Knit surfaces" & " " & swMirrorSolidFeatureData.KnitSurface)  
  
        'Roll back to get to the bodies
```

```
status = swMirrorSolidFeatureData.AccessSelections(swModel, Nothing)
swSelectionMgr = swModel.SelectionManager
swSelData = swSelectionMgr.CreateSelectData
bodies = swMirrorSolidFeatureData.PatternBodyArray
For i = 0 To UBound(bodies)
    swBody = bodies(i)
    status = swBody.Select(True, 0)
    Debug.Print("    Body " & i + 1 & "'s type (solid body = 0) = " & swBody.GetType)
Next i

'Release selection access
swMirrorSolidFeatureData.ReleaseSelectionAccess()
```

End Sub

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

End Class

Get Model Dependencies Example (C++ COM)

This example shows how to get the names of the dependencies for a model.

```
// -----  
void APITestFunction()  
{  
    HRESULT                hr = S_OK;  
    VARIANT_BOOL            bRet = VARIANT_FALSE;  
    long                    nRetVal = -1;  
    CComPtr<ISldWorks>      pSldWorks;  
    CComPtr<IModelDoc2>     pModelDoc;  
    long                    nNumDepend = -1;  
    BSTR*                   psDepends = NULL;  
    long                    i = -1;  
    // Connect to SOLIDWORKS Design application  
    pSldWorks = TheApplication->GetSWApp();  
    ASSERT(pSldWorks);  
    hr = pSldWorks->get_IActiveDoc2(&pModelDoc);  
    ASSERT(pModelDoc);  
    hr = pModelDoc->IGetNumDependencies(VARIANT_TRUE, VARIANT_TRUE, &nNumDepend);  
    ASSERT(nNumDepend > 0);  
    psDepends = new BSTR[nNumDepend];  
    ASSERT(psDepends);  
    ZeroMemory(psDepends, nNumDepend * sizeof(BSTR));  
    hr = pModelDoc->IGetDependencies2(VARIANT_TRUE, VARIANT_TRUE, VARIANT_FALSE, psDepends);  
    ASSERT(psDepends);  
    for (i = 0; i < nNumDepend / 2; i++)  
    {  
        CComBSTR            sDepend1;  
        CComBSTR            sDepend2;  
        CComBSTR            sOutputStr;  
        sDepend1.Attach(psDepends[2 * i + 0]);  
        sDepend2.Attach(psDepends[2 * i + 1]);  
        sOutputStr = sDepend1;  
        sOutputStr += CComBSTR(_T(" --> "));  
        sOutputStr += sDepend2;  
        sOutputStr += CComBSTR(_T("\n"));  
        OutputDebugString(sOutputStr);  
    }  
    delete [] psDepends;  
    return;  
} //void APITestFunction()  
// -----
```

Get Model Material Property Values Example (C#)

This example shows how to get the material property values of components in lightweight mode.

```
//-----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\api\TopAssembly1.SLDASM in lightweight  
//    mode.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Gets the material property values of the lightweight components in the  
//    model.  
// 2. Inspect the Immediate window.  
//  
// NOTE: Because the model is used elsewhere, do not save changes.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
namespace GetMatPropValues_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
  
        ModelDoc2 swDoc;  
        AssemblyDoc swADoc;  
        object[] varComp;  
        double[] varMatProp;  
  
        public void Main()  
        {  
            swDoc = (ModelDoc2)swApp.ActiveDoc;  
            swADoc = (AssemblyDoc)swDoc;  
            varComp = (object[])swADoc.GetComponents(true);  
            int I = 0;  
            for (I = 0; I < varComp.Length; I++)  
            {  
                Component2 swComp = default(Component2);  
                swComp = (Component2)varComp[I];  
  
                varMatProp =  
(double[])swComp.GetModelMaterialPropertyValues(swComp.ReferencedConfiguration);  
                if (!((varMatProp == null)))  
                {  
                    Debug.Print(swComp.Name2 + "(" + I + ")" + "ConfigName : " +  
swComp.ReferencedConfiguration + "MatProp : ");  
                    Debug.Print("Red: " + (varMatProp[0]) * 255.0);  
                    Debug.Print("Green: " + (varMatProp[1]) * 255.0);  
                    Debug.Print("Blue: " + (varMatProp[2]) * 255.0);  
                    Debug.Print("Ambient: " + (varMatProp[3]) * 100.0 + "%");  
                    Debug.Print("Diffuse: " + (varMatProp[4]) * 100.0 + "%");  
                    Debug.Print("Specularity: " + (varMatProp[5]) * 100.0 + "%");  
                    Debug.Print("Shininess: " + (varMatProp[6]) * 100.0 + "%");  
                    Debug.Print("Transparency: " + (varMatProp[7]) * 100.0 + "%");  
                    Debug.Print("Emission: " + (varMatProp[8]) * 100.0 + "%");  
                }  
            }  
        }  
    }  
}
```

```
        Debug.Print("");
    }
    varComp = null;
}

public Sldworks swApp;
}
}
```

Get Model Material Property Values Example (VBA)

This example shows how to get the material property values of components in lightweight mode.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\TopAssembly1.SLDASM in lightweight  
'    mode.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the material property values of the lightweight components in the  
'    model.  
' 2. Inspect the Immediate window.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
' -----  
  
Dim swApp As SldWorks.SldWorks  
Dim swDoc As SldWorks.ModelDoc2  
Dim swADoc As SldWorks.AssemblyDoc  
Dim varComp As Variant  
Dim varMatProp As Variant  
  
Option Explicit  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swDoc = swApp.ActiveDoc  
Set swADoc = swDoc  
varComp = swADoc.GetComponents(True)  
Dim I As Long  
For I = LBound(varComp) To UBound(varComp)  
    Dim swComp As SldWorks.Component2  
    Set swComp = varComp(I)  
  
    varMatProp = swComp.GetModelMaterialPropertyValues(swComp.ReferencedConfiguration)  
    If Not (IsEmpty(varMatProp)) Then  
        Debug.Print (swComp.Name2 & "(" & I & ")" & "ConfigName : " &  
swComp.ReferencedConfiguration & "MatProp : ")  
        Debug.Print "Red: " & (varMatProp(0)) * 255#  
        Debug.Print "Green: " & (varMatProp(1)) * 255#  
        Debug.Print "Blue: " & (varMatProp(2)) * 255#  
        Debug.Print "Ambient: " & (varMatProp(3)) * 100# & "%"  
        Debug.Print "Diffuse: " & (varMatProp(4)) * 100# & "%"  
        Debug.Print "Specularity: " & (varMatProp(5)) * 100# & "%"  
        Debug.Print "Shininess: " & (varMatProp(6)) * 100# & "%"  
        Debug.Print "Transparency: " & (varMatProp(7)) * 100# & "%"  
        Debug.Print "Emission: " & (varMatProp(8)) * 100# & "%"  
    End If  
  
    Debug.Print ""  
Next I  
varComp = Empty  
End Sub
```

Get Model Material Property Values Example (VB.NET)

This example shows how to get the material property values of components in lightweight mode.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\TopAssembly1.SLDASM in lightweight  
'    mode.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the material property values of the lightweight components in the  
'    model.  
' 2. Inspect the Immediate window.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
' -----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swDoc As ModelDoc2  
    Dim swADoc As AssemblyDoc  
    Dim varComp As Object  
    Dim varMatProp As Object
```

```
    Sub main()
```

```
        swDoc = swApp.ActiveDoc  
        swADoc = swDoc  
        varComp = swADoc.GetComponents(True)  
        Dim I As Long  
        For I = LBound(varComp) To UBound(varComp)  
            Dim swComp As Component2  
            swComp = varComp(I)  
  
            varMatProp =  
swComp.GetModelMaterialPropertyValues(swComp.ReferencedConfiguration)  
            If Not (IsNothing(varMatProp)) Then  
                Debug.Print(swComp.Name2 & "(" & I & ")" & "ConfigName : " &  
swComp.ReferencedConfiguration & "MatProp : ")  
                Debug.Print("Red: " & (varMatProp(0)) * 255.0#)  
                Debug.Print("Green: " & (varMatProp(1)) * 255.0#)  
                Debug.Print("Blue: " & (varMatProp(2)) * 255.0#)  
                Debug.Print("Ambient: " & (varMatProp(3)) * 100.0# & "%")  
                Debug.Print("Diffuse: " & (varMatProp(4)) * 100.0# & "%")  
                Debug.Print("Specularity: " & (varMatProp(5)) * 100.0# & "%")  
                Debug.Print("Shininess: " & (varMatProp(6)) * 100.0# & "%")  
                Debug.Print("Transparency: " & (varMatProp(7)) * 100.0# & "%")  
                Debug.Print("Emission: " & (varMatProp(8)) * 100.0# & "%")  
            End If  
  
            Debug.Print("")  
        Next I  
        varComp = Nothing
```

End Sub

Public swApp As SldWorks

End Class

Get Model Pathnames from a BOM Table Example (C#)

This example shows how to get the model pathnames from a specified row of a BOM table annotation.

```
//-----
// Preconditions:
// 1. Open a drawing containing one BOM table annotation named Bill of Materials1.
// 2. Modify the namespace to match the name of your C# project.
// 3. Open an Immediate Window.
//
// Postconditions: Inspect the Immediate Window for model pathnames.
// -----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
namespace ModelPathNames_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDraw = default(DrawingDoc);
            Feature swFeat = default(Feature);
            BomFeature swBomFeat = default(BomFeature);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDraw = (DrawingDoc)swModel;
            swFeat = (Feature)swModel.FirstFeature();
            while ((swFeat != null))
            {
                if ((swFeat.GetTypeName() == "BomFeat"))
                {
                    Debug.Print("*****");
                    Debug.Print("Feature Name = " + swFeat.Name);
                    swBomFeat = (BomFeature)swFeat.GetSpecificFeature2();
                    if ((swFeat.Name == "Bill of Materials1"))
                    {
                        ProcessBomFeature(swApp, swModel, swBomFeat);
                    }
                    swFeat = (Feature)swFeat.GetNextFeature();
                }
            }
            public void ProcessBomFeature(SldWorks swApp, ModelDoc2 swModel, BomFeature
swBomFeat)
            {
                GetPathNames(swApp, swBomFeat);
            }

            public void GetPathNames(SldWorks swApp, BomFeature swBomFeature)
            {
                String[] vConfigurations = null;
```

```

string strConfiguration = null;
Object vVisibility = null;
int lIdx = 0;
int lNumConfigurations = 0;
int lNumRow = 0;
int lNumColumn = 0;
int lRow = 0;
Object[] swBOMTableAnnotationArray = null;
BomTableAnnotation swBOMTableAnnotation = default(BomTableAnnotation);
TableAnnotation swTableAnnotation = default(TableAnnotation);
ModelDoc2 swDocument = default(ModelDoc2);
AssemblyDoc swAssembly = default(AssemblyDoc);
string strDocumentName = null;
int lStartRow = 0;
bool bGetVisible = false;
String[] vModelPathNames = null;
string strItemNumber = "";
string strPartNumber = "";
object vModelPathName = null;
string strModelPathName = null;

strDocumentName = swBomFeature.GetReferencedModelName();
swDocument = (ModelDoc2)swApp.GetOpenDocumentByName(strDocumentName);
Debug.Print("Referenced model = " + strDocumentName);
swAssembly = (AssemblyDoc)swDocument;

swBOMTableAnnotationArray = (Object[])swBomFeature.GetTableAnnotations();
// Get the only BOM table annotation
swBOMTableAnnotation = (BomTableAnnotation)swBOMTableAnnotationArray[0];
swTableAnnotation = (TableAnnotation)swBOMTableAnnotation;
lNumRow = swTableAnnotation.RowCount;
lNumColumn = swTableAnnotation.ColumnCount;

lStartRow = 1;

if ((!(swTableAnnotation.TitleVisible == false)))
{
    lStartRow = 2;
}

bGetVisible = false;
Debug.Print("# configurations = " +
swBomFeature.GetConfigurationCount(bGetVisible));
vConfigurations = (String[])swBomFeature.GetConfigurations(bGetVisible, ref
vVisibility);

if ((vConfigurations != null))
{
    lNumConfigurations = vConfigurations.GetUpperBound(0) -
vConfigurations.GetLowerBound(0) + 1;
    for (lIdx = 0; lIdx <= (lNumConfigurations - 1); lIdx++)
    {
        strConfiguration = vConfigurations[lIdx];
        Debug.Print("");
        Debug.Print("Configuration: " + strConfiguration);

        for (lRow = lStartRow; lRow <= (lNumRow - 1); lRow++)
        {
            Debug.Print(" row = " + (lRow - lStartRow + 1));
            vModelPathNames =
(String[])swBOMTableAnnotation.GetModelPathNames(lRow, out strItemNumber, out
strPartNumber);

            Debug.Print(" item number = " + strItemNumber);
            Debug.Print(" part number = " + strPartNumber);

```

```

        if (((vModelPathNames != null)))
        {
            Debug.Print("    # models contributing to row = " +
swBOMTableAnnotation.GetModelPathNamesCount(lRow));
            foreach (String vModelPathName_loopVariable in
vModelPathNames)
            {
                vModelPathName = vModelPathName_loopVariable;
                strModelPathName = (String)vModelPathName;
                Debug.Print("        " + strModelPathName);
            }
        }
        else
        {
            Debug.Print("    # models contributing to row = 0");
            Debug.Assert(false);
        }
    }
}

public SldWorks swApp;
}
}

```

Get Model Pathnames from a BOM Table Example (VBA)

This example shows how to get the model pathnames from a specified row of a BOM table annotation.

```
' -----  
' Preconditions:  
' 1. Open a drawing containing a BOM table annotation named Bill of Materials1.  
' 2. Open an Immediate Window.  
'  
' Postconditions: Inspect the Immediate Window for model pathnames.  
' -----
```

Option Explicit

```
Sub Main()  
    Dim swApp      As SldWorks.SldWorks  
    Dim swModel    As SldWorks.ModelDoc2  
    Dim swDraw     As SldWorks.DrawingDoc  
    Dim swFeat     As SldWorks.Feature  
    Dim swBomFeat  As SldWorks.BomFeature  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swDraw = swModel  
    Set swFeat = swModel.FirstFeature  
    Do While Not swFeat Is Nothing  
        If (swFeat.GetTypeName = "BomFeat") Then  
            Debug.Print "*****"  
            Debug.Print "Feature Name = " & swFeat.Name  
            Set swBomFeat = swFeat.GetSpecificFeature2  
            If (swFeat.Name = "Bill of Materials1") Then  
                ProcessBomFeature swApp, swModel, swBomFeat  
            End If  
        End If  
        Set swFeat = swFeat.GetNextFeature  
    Loop  
End Sub  
Sub ProcessBomFeature(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swBomFeat  
As SldWorks.BomFeature)  
    GetPathNames swApp, swBomFeat  
End Sub  
  
Sub GetPathNames(swApp As SldWorks.SldWorks, ByVal swBomFeature As SldWorks.BomFeature)  
    Dim vConfigurations As Variant  
    Dim strConfiguration As String  
    Dim vVisibility As Variant  
    Dim lIdx As Long  
    Dim lNumConfigurations As Long  
    Dim lNumRow As Long  
    Dim lNumColumn As Long  
    Dim lRow As Long  
    Dim swBOMTableAnnotation As SldWorks.BomTableAnnotation  
    Dim swTableAnnotation As SldWorks.TableAnnotation  
    Dim swDocument As SldWorks.ModelDoc2  
    Dim swAssembly As SldWorks.AssemblyDoc  
    Dim strDocumentName As String  
    Dim lStartRow As Long  
    Dim bGetVisible As Boolean  
    Dim vModelPathNames As Variant
```

```

Dim strItemNumber           As String
Dim strPartNumber           As String
Dim vModelPathName          As Variant
Dim strModelPathName        As String

strDocumentName = swBomFeature.GetReferencedModelName
Set swDocument = swApp.GetOpenDocumentByName(strDocumentName)
Debug.Print "Referenced model = " & strDocumentName
Set swAssembly = swDocument

' For now assume we have only 1 table annotation.
Set swBOMTableAnnotation = swBomFeature.GetTableAnnotations(0)
Set swTableAnnotation = swBOMTableAnnotation
lNumRow = swTableAnnotation.RowCount
lNumColumn = swTableAnnotation.ColumnCount

lStartRow = 1

If (Not (swTableAnnotation.TitleVisible = False)) Then
    lStartRow = 2
End If

bGetVisible = False
Debug.Print "# configurations = " & swBomFeature.GetConfigurationCount(bGetVisible)
vConfigurations = swBomFeature.GetConfigurations(bGetVisible, vVisibility)

If (Not IsEmpty(vConfigurations)) Then
    lNumConfigurations = UBound(vConfigurations) - LBound(vConfigurations) + 1
    For lIdx = 0 To (lNumConfigurations - 1)
        strConfiguration = vConfigurations(lIdx)
        Debug.Print ""
        Debug.Print "Configuration: " & strConfiguration

        For lRow = lStartRow To (lNumRow - 1)
            Debug.Print " row = " & (lRow - lStartRow + 1)
            vModelPathNames = swTableAnnotation.GetModelPathNames(lRow, strItemNumber,
strPartNumber)

            Debug.Print " item number = " & strItemNumber
            Debug.Print " part number = " & strPartNumber

            If (Not IsEmpty(vModelPathNames)) Then
                Debug.Print " # models contributing to row = " &
swTableAnnotation.GetModelPathNamesCount(lRow)
                For Each vModelPathName In vModelPathNames
                    strModelPathName = vModelPathName
                    Debug.Print " " & strModelPathName
                Next vModelPathName
            Else
                Debug.Print " # models contributing to row = 0"
                Debug.Assert (False)
            End If
        Next lRow
    Next lIdx
End If
End Sub

```

Get Model Pathnames from a BOM Table Example (VB.NET)

This example shows how to get the model pathnames from a specified row of a BOM table annotation.

```
' -----  
' Preconditions:  
' 1. Open a drawing containing a BOM table annotation named Bill of Materials1.  
' 2. Open an Immediate Window.  
'  
' Postconditions: Inspect the Immediate Window for model pathnames.  
' -----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub Main()
```

```
        Dim swModel As ModelDoc2  
        Dim swDraw As DrawingDoc  
        Dim swFeat As Feature  
        Dim swBomFeat As BomFeature
```

```
        swModel = swApp.ActiveDoc  
        swDraw = swModel  
        swFeat = swModel.FirstFeature  
        Do While Not swFeat Is Nothing  
            If (swFeat.GetTypeName = "BomFeat") Then  
                Debug.Print("*****")  
                Debug.Print("Feature Name = " & swFeat.Name)  
                swBomFeat = swFeat.GetSpecificFeature2  
                If (swFeat.Name = "Bill of Materials1") Then  
                    ProcessBomFeature(swApp, swModel, swBomFeat)  
                End If  
            End If  
            swFeat = swFeat.GetNextFeature  
        Loop
```

```
    End Sub
```

```
    Sub ProcessBomFeature(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal  
swBomFeat As BomFeature)  
        GetPathNames(swApp, swBomFeat)  
    End Sub
```

```
    Sub GetPathNames(ByVal swApp As SldWorks, ByVal swBomFeature As BomFeature)  
        Dim vConfigurations As Object  
        Dim strConfiguration As String  
        Dim vVisibility As Object = Nothing  
        Dim lIdx As Long  
        Dim lNumConfigurations As Long  
        Dim lNumRow As Long  
        Dim lNumColumn As Long  
        Dim lRow As Long  
        Dim swBOMTableAnnotation As BomTableAnnotation  
        Dim swTableAnnotation As TableAnnotation  
        Dim swDocument As ModelDoc2  
        Dim swAssembly As AssemblyDoc
```

```

Dim strDocumentName As String
Dim lStartRow As Long
Dim bGetVisible As Boolean
Dim vModelPathNames As Object
Dim strItemNumber As String = ""
Dim strPartNumber As String = ""
Dim vModelPathName As Object
Dim strModelPathName As String

strDocumentName = swBomFeature.GetReferencedModelName
swDocument = swApp.GetOpenDocumentByName(strDocumentName)
Debug.Print("Referenced model = " & strDocumentName)
swAssembly = swDocument

' For now assume we have only 1 table annotation.
swBOMTableAnnotation = swBomFeature.GetTableAnnotations(0)
swTableAnnotation = swBOMTableAnnotation
lNumRow = swTableAnnotation.RowCount
lNumColumn = swTableAnnotation.ColumnCount

lStartRow = 1

If (Not (swTableAnnotation.TitleVisible = False)) Then
    lStartRow = 2
End If

bGetVisible = False
Debug.Print("# configurations = " &
swBomFeature.GetConfigurationCount(bGetVisible))
vConfigurations = swBomFeature.GetConfigurations(bGetVisible, vVisibility)

If (Not IsNothing(vConfigurations)) Then
    lNumConfigurations = UBound(vConfigurations) - LBound(vConfigurations) + 1
    For lIdx = 0 To (lNumConfigurations - 1)
        strConfiguration = vConfigurations(lIdx)
        Debug.Print("")
        Debug.Print("Configuration: " & strConfiguration)

        For lRow = lStartRow To (lNumRow - 1)
            Debug.Print(" row = " & (lRow - lStartRow + 1))
            vModelPathNames = swTableAnnotation.GetModelPathNames(lRow,
strItemNumber, strPartNumber)

            Debug.Print(" item number = " & strItemNumber)
            Debug.Print(" part number = " & strPartNumber)

            If (Not IsNothing(vModelPathNames)) Then
                Debug.Print(" # models contributing to row = " &
swTableAnnotation.GetModelPathNamesCount(lRow))
                For Each vModelPathName In vModelPathNames
                    strModelPathName = vModelPathName
                    Debug.Print(" " & strModelPathName)
                Next vModelPathName
            Else
                Debug.Print(" # models contributing to row = 0")
                Debug.Assert(False)
            End If
        Next lRow
    Next lIdx
End If
End Sub

Public swApp As SldWorks

End Class

```

Get Model Textures Example (C#)

This example shows how to get the texture applied to components that are displayed in lightweight mode.

```
//-----  
// Preconditions:  
// 1. Open in lightweight mode public_documents\samples\tutorial\api\TopAssembly1.SLDASM  
// 2. Open the Immediate window.  
//  
// Postconditions: Inspect the Immediate window for the textures of the  
// lightweight components in the model.  
//  
// NOTE: Because the model is used elsewhere, do not save changes.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
namespace GetModelTexture_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
  
        ModelDoc2 swDoc;  
        AssemblyDoc swADoc;  
        object varComp;  
        Object[] compArray;  
        Texture swTexture;  
  
        public void Main()  
        {  
            swDoc = (ModelDoc2)swApp.ActiveDoc;  
            swADoc = (AssemblyDoc)swDoc;  
            varComp = swADoc.GetComponents(true);  
            compArray = (Object[])varComp;  
  
            int I = 0;  
            string compName = null;  
            for (I = 0; I < compArray.Length; I++)  
            {  
                Component2 swComp = default(Component2);  
                swComp = (Component2)compArray[I];  
                compName = swComp.ReferencedConfiguration;  
  
                swTexture = swComp.GetModelTexture(compName);  
                if ((swTexture != null))  
                {  
                    Debug.Print(swComp.Name2 + "(" + I + ")" + "ConfigName : " + compName  
+ "MatProp : ");  
                    Debug.Print(swTexture.MaterialName);  
                    Debug.Print("");  
                }  
            }  
            varComp = null;  
        }  
    }  
}
```

```
        public Sldworks swApp;  
    }  
}
```

Get Model Textures Example (VBA)

This example shows how to get the texture applied to components that are displayed in lightweight mode.

```
' -----  
' Preconditions:  
' 1. Open in lightweight mode public_documents\samples\tutorial\api\TopAssembly1.SLDASM.  
' 2. Open the Immediate window.  
'  
' Postconditions: Inspect the Immediate window for the textures of the  
' lightweight components in the model.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swDoc As SldWorks.ModelDoc2  
Dim swADoc As SldWorks.AssemblyDoc  
Dim varComp As Variant  
Dim swTexture As SldWorks.Texture
```

Sub main()

```
    Set swApp = Application.SldWorks  
    Set swDoc = swApp.ActiveDoc  
    Set swADoc = swDoc  
    varComp = swADoc.GetComponents(True)  
    Dim I As Long  
    Dim compName As String  
    For I = LBound(varComp) To UBound(varComp)  
        Dim swComp As SldWorks.Component2  
        Set swComp = varComp(I)  
        compName = swComp.ReferencedConfiguration  
  
        Set swTexture = swComp.GetModelTexture(compName)  
        If Not (swTexture Is Nothing) Then  
            Debug.Print (swComp.Name2 & "(" & I & ")" & "ConfigName : " & compName &  
"MatProp : ")  
            Debug.Print (swTexture.MaterialName)  
            Debug.Print ""  
  
        End If  
    Next I  
    varComp = Empty
```

```
End Sub
```

Get Model Textures Example (VB.NET)

This example shows how to get the texture applied to components that are displayed in lightweight mode.

```
' -----  
' Preconditions:  
' 1. Open in lightweight mode public_documents\samples\tutorial\api\TopAssembly1.SLDASM  
' 2. Open the Immediate window.  
'  
' Postconditions: Inspect the Immediate window for the textures of the  
' lightweight components in the model.  
'  
' NOTE: Because the model is used elsewhere, do not save changes.  
' -----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swDoc As ModelDoc2
```

```
    Dim swADoc As AssemblyDoc
```

```
    Dim varComp As Object
```

```
    Dim swTexture As Texture
```

```
    Sub main()
```

```
        swDoc = swApp.ActiveDoc
```

```
        swADoc = swDoc
```

```
        varComp = swADoc.GetComponents(True)
```

```
        Dim I As Long
```

```
        Dim compName As String
```

```
        For I = LBound(varComp) To UBound(varComp)
```

```
            Dim swComp As Component2
```

```
            swComp = varComp(I)
```

```
            compName = swComp.ReferencedConfiguration
```

```
            swTexture = swComp.GetModelTexture(compName)
```

```
            If Not (swTexture Is Nothing) Then
```

```
                Debug.Print(swComp.Name2 & "(" & I & ")" & "ConfigName : " & compName &
```

```
"MatProp : ")
```

```
                Debug.Print(swTexture.MaterialName)
```

```
                Debug.Print("")
```

```
            End If
```

```
        Next I
```

```
        varComp = Nothing
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Get Motion Study Example (VBA)

This example shows how to get a Physical Simulation motion study and its properties.

```
' -----  
' Preconditions:  
' 1. Open a model that has a Physical Simulation motion study  
'    named PhysSim.  
' 2. In SOLIDWORKS Design, load the SOLIDWORKS Motion Study add-in  
'    (click Tools > Add-Ins > SOLIDWORKS Motion).  
' 3. In the SOLIDWORKS VBA IDE, add a reference to the  
'    SolidWorks MotionStudy type library.  
' 4. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the MotionManager.  
' 2. Gets and activates the PhysSim motion study.  
' 3. Gets some motion study properties and features.  
' 4. Plays the motion study.  
' 5. In SOLIDWORKS Design, click Stop in the MotionManager  
'    to stop playing the motion study.  
' 6. Gets the motion study results and whether the  
'    results are out of date.  
' 7. Examine the Immediate window.  
' -----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swMotionMgr As SwMotionStudy.MotionStudyManager  
Dim swMotionStudy1 As SwMotionStudy.MotionStudy  
Dim swResults As SwMotionStudy.MotionStudyResults  
Dim swMotionStudyProps As SwMotionStudy.MotionStudyProperties  
Dim swPhysicalSimProps As SwMotionStudy.PhysicalSimulationMotionStudyProperties  
Dim boolstatus As Boolean  
Dim i As Long  
Dim vMotionFeatures As Variant  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
  
    ' Get the MotionManager  
    Set swMotionMgr = swModelDocExt.GetMotionStudyManager()  
    If (swMotionMgr Is Nothing) Then  
        End  
    End If  
  
    ' Get motion study named PhysSim  
    Set swMotionStudy1 = swMotionMgr.GetMotionStudy("PhysSim")  
    If (swMotionStudy1 Is Nothing) Then  
        MsgBox "PhysSim is not available."  
        End  
    End If  
  
    swMotionStudy1.StudyType = swMotionStudyType_e.swMotionStudyTypePhysicalSimulation  
  
    'Active the motion study  
    If Not swMotionStudy1.IsActive Then swMotionStudy1.Activate  
    Debug.Print "Duration of motion study: " & swMotionStudy1.GetDuration  
    Debug.Print "Timebar position on timeline: " & swMotionStudy1.GetTime
```

```

Debug.Print "Number of motion features: " & swMotionStudy1.GetMotionFeaturesCount
Set swMotionStudyProps = swMotionStudy1.GetProperties(2)
Set swPhysicalSimProps = swMotionStudyProps
Debug.Print ""
Debug.Print "Physical simulation properties: "
Debug.Print "    Frame rate: " & swMotionStudyProps.GetFrameRate
Debug.Print "    Geometric accuracy: " & swPhysicalSimProps.GeometricAccuracy
Debug.Print "    Mate accuracy: " & swPhysicalSimProps.MateAccuracy
swPhysicalSimProps.MateAccuracy = 1
vMotionFeatures = swMotionStudy1.GetMotionFeatures
For i = 0 To UBound(vMotionFeatures)
Dim swFeat As Feature
    Set swFeat = vMotionFeatures(i)
    PhysicalSimFeatureTests swFeat, swModel
Next i

'Calculate
swMotionStudy1.Calculate
WaitForCaluculation swMotionStudy1
Set swResults = swMotionStudy1.GetResults(2)
Debug.Print ""
Debug.Print "Are the motion study results out of date? " & swResults.IsOutOfDate

End Sub

Private Sub WaitForCaluculation(SwMotionStudy As MotionStudy)
    Do While SwMotionStudy.IsPlaying
        DoEvents
    Loop
End Sub

Private Sub PhysicalSimFeatureTests(swFeat As Feature, TopDoc As ModelDoc2)
    Dim SimTypeName As String
    Dim SimulationGravityFeatureData As SldWorks.SimulationGravityFeatureData

    SimTypeName = swFeat.GetType
    Debug.Print ""
    ' Print the name of the type of simulation and its gravity strength
    Select Case SimTypeName
        Case "AEMGravity"
            Debug.Print "    Type of feature: AEMGravity"
        Case "AEMLinearMotor"
            Debug.Print "    Type of feature: AEMLinearMotor"
        Case "AEMRotationalMotor"
            Debug.Print "    Type of feature: AEMRotationalMotor"
        Case "AEMLinearSpring"
            Debug.Print "    Type of feature: AEMLinearSpring"
        Case Else
            Debug.Print "    Type of feature: " & SimTypeName
            MsgBox SimTypeName
    End Select
End Sub

```

Get Multi-jog Leader Data Example (C#)

This example shows how to get data for a multi-jog leader.

```
//-----  
// Preconditions:  
// 1. Open a drawing document with at least one multi-jog leader.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Selects a multi-jog leader.  
// 2. Gets data for the multi-jog leader.  
// 3. Examine the Immediate window.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace MultiJogLeaderCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            DrawingDoc swDrawingDoc = default(DrawingDoc);  
            Sheet swSheet = default(Sheet);  
            View swView = default(View);  
            object obj = null;  
            Annotation swAnnotation = default(Annotation);  
            MultiJogLeader swMultiJogLeader = default(MultiJogLeader);  
            int nbrLines = 0;  
            double[] lineData = null;  
            int lineType = 0;  
            double startPointX = 0;  
            double startPointY = 0;  
            double startPointZ = 0;  
            double endPointX = 0;  
            double endPointY = 0;  
            double endPointZ = 0;  
            int i = 0;  
  
            //Get multi-jog leader  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swDrawingDoc = (DrawingDoc)swModel;  
            swSheet = (Sheet)swDrawingDoc.GetCurrentSheet();  
            Debug.Print("Sheet name: " + swSheet.GetName());  
            //First view is Sheet, so get next view  
            swView = (View)swDrawingDoc.GetFirstView();  
            swView = (View)swView.GetNextView();  
            Debug.Print("View name: " + swView.GetName2());  
            obj = (object)swView.GetFirstMultiJogLeader2();  
            swMultiJogLeader = (MultiJogLeader)obj;  
  
            //Get multi-jog leader data  
            swAnnotation = (Annotation)swMultiJogLeader.GetAnnotation();  
            Debug.Print("Annotation name: " + swAnnotation.GetName());  
            Debug.Print("Type of annotation (11 = multi-jog leader): " + swAnnotation.GetType());  
            nbrLines = swMultiJogLeader.GetLineCount();  
            Debug.Print("Number of lines in multi-jog leader: " + nbrLines);  
            lineData = (double[])swMultiJogLeader.GetLineAtIndex(1);  
            for (i = 0; i <= nbrLines - 1; i++)  
            {  
                if ((lineData != null))  
                {  
                    Debug.Print(" Line " + (i + 1) + ":");  
                    lineType = (int)lineData[0];  
                    startPointX = lineData[1];  
                    startPointY = lineData[2];  
                    startPointZ = lineData[3];  
                    endPointX = lineData[4];  
                }  
            }  
        }  
    }  
}
```

```

        endPointY = lineData[5];
        endPointZ = lineData[6];
        Debug.Print(" Line type as defined by swLineTypes_e: " + lineType);
        Debug.Print(" Line x, y, and z start points: " + startPointX + ", " + startPointY +
", and " + startPointZ);
        Debug.Print(" Line x, y, and z end points: " + endPointX + ", " + endPointY + ", and
" + endPointZ);
    }
    }
    Debug.Print("Number of arrow heads in multi-
jog leader: " + swMultiJogLeader.GetArrowHeadCount());

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

```

Get Multi-jog Leader Data Example (VBA)

This example shows how to get data for a multi-jog leader.

```
' -----  
' Preconditions:  
' 1. Open a drawing document with at least one multi-jog leader.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Selects a multi-jog leader.  
' 2. Gets data for the multi-jog leader.  
' 3. Examine the Immediate window.  
' -----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swDrawingDoc As SldWorks.DrawingDoc  
Dim swSheet As SldWorks.Sheet  
Dim swView As SldWorks.View  
Dim obj As Object  
Dim swAnnotation As SldWorks.Annotation  
Dim swMultiJogLeader As SldWorks.MultiJogLeader  
Dim nbrLines As Long  
Dim lineData As Variant  
Dim i As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
  
    'Get multi-jog leader  
    Set swModel = swApp.ActiveDoc  
    Set swDrawingDoc = swModel  
    Set swSheet = swDrawingDoc.GetCurrentSheet  
    Debug.Print "Sheet name: " & swSheet.GetName  
    'First view is Sheet, so get next view  
    Set swView = swDrawingDoc.GetFirstView  
    Set swView = swView.GetNextView  
  
    Debug.Print "View name: " & swView.GetName2  
    Set obj = swView.GetFirstMultiJogLeader2  
    Set swMultiJogLeader = obj  
  
    'Get multi-jog leader data  
    Set swAnnotation = swMultiJogLeader.GetAnnotation  
    Debug.Print "Annotation name: " & swAnnotation.GetName  
    Debug.Print "Type of annotation (11 = multi-jog leader): " & swAnnotation.GetType  
    nbrLines = swMultiJogLeader.GetLineCount  
    Debug.Print "Number of lines in multi-jog leader: " & nbrLines  
    lineData = swMultiJogLeader.GetLineAtIndex(1)  
    For i = 0 To nbrLines - 1  
        If Not IsEmpty(lineData) Then  
            Debug.Print " Line " & i + 1 & ":"  
            Debug.Print " Line type as defined by swLineTypes_e: " & lineData(0)  
            Debug.Print " Line x, y, and z start points: " & lineData(1) & ", " & lineData(2) & ", and "  
& lineData(3)  
            Debug.Print " Line x, y, and z end points: " & lineData(4) & ", " & lineData(5) & ", and " &  
lineData(6)  
            End If  
        Next i  
  
        Debug.Print "Number of arrow heads in multi-jog leader: " & swMultiJogLeader.GetArrowHeadCount
```

End Sub

Get Multi-jog Leader Data Example (VB.NET)

This example shows how to get data for a multi-jog leader.

```
'-----
' Preconditions:
' 1. Open a drawing document with at least one multi-jog leader.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Selects a multi-jog leader.
' 2. Gets data for the multi-jog leader.
' 3. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swDrawingDoc As DrawingDoc
        Dim swSheet As Sheet
        Dim swView As View
        Dim obj As Object
        Dim swAnnotation As Annotation
        Dim swMultiJogLeader As MultiJogLeader
        Dim nbrLines As Integer
        Dim lineData As Object
        Dim lineType As Integer
        Dim startPointX As Double
        Dim startPointY As Double
        Dim startPointZ As Double
        Dim endPointX As Double
        Dim endPointY As Double
        Dim endPointZ As Double
        Dim i As Integer

        'Get multi-jog leader
        swModel = swApp.ActiveDoc
        swDrawingDoc = swModel
        swSheet = swDrawingDoc.GetCurrentSheet
        Debug.Print("Sheet name: " & swSheet.GetName)
        'First view is Sheet, so get next view
        swView = swDrawingDoc.GetFirstView
        swView = swView.GetNextView
        Debug.Print("View name: " & swView.GetName2)
        obj = swView.GetFirstMultiJogLeader2
        swMultiJogLeader = obj

        'Get multi-jog leader data
        swAnnotation = swMultiJogLeader.GetAnnotation
        Debug.Print("Annotation name: " & swAnnotation.GetName)
        Debug.Print("Type of annotation (11 = multi-jog leader): " & swAnnotation.GetType)
        nbrLines = swMultiJogLeader.GetLineCount
        Debug.Print("Number of lines in multi-jog leader: " & nbrLines)
        lineData = swMultiJogLeader.GetLineAtIndex(1)
        For i = 0 To nbrLines - 1
            If Not lineData Is Nothing Then
                Debug.Print(" Line " & i + 1 & ":")
                lineType = lineData(0)
                startPointX = lineData(1)
                startPointY = lineData(2)
                startPointZ = lineData(3)
                endPointX = lineData(4)
                endPointY = lineData(5)
                endPointZ = lineData(6)
                Debug.Print(" Line type as defined by swLineTypes_e: " & lineType)
                Debug.Print(" Line x, y, and z start points: " & startPointX & ", " & startPointY & ", a
```

```
nd " & startPointZ)
    Debug.Print("  Line x, y, and z end points: " & endPointX & ", " & endPointY & ", and " &
endPointZ)
End If
Next i
Debug.Print("Number of arrow heads in multi-jog leader: " & swMultiJogLeader.GetArrowHeadCount)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class
```

Get Name of Drawing Sheet Template Example (VBA)

This example shows how to get the name of a drawing sheet template.

```
' -----  
' Preconditions:  
' 1. Open a drawing document that contains at least one sheet.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the current sheet, drawing's path and file name,  
'    sheet's name, template's path and file name, and sheet's  
'    properties.  
' 2. Examine the Immediate window.  
' -----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swDraw As SldWorks.DrawingDoc  
    Dim swSheet As SldWorks.Sheet  
    Dim vSheetProps As Variant  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swDraw = swModel  
    Set swSheet = swDraw.GetCurrentSheet  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "  Sheet = " & swSheet.GetName  
    Debug.Print "    Template = " & swSheet.GetTemplateName  
  
    ' Get current sheet settings  
    vSheetProps = swSheet.GetProperties  
    Debug.Print "      PaperSize      = " & vSheetProps(0)  
    Debug.Print "      TemplateIn     = " & vSheetProps(1)  
    Debug.Print "      Scale1          = " & vSheetProps(2)  
    Debug.Print "      scale2          = " & vSheetProps(3)  
    Debug.Print "      FirstAngle      = " & vSheetProps(4)  
    Debug.Print "      Width           = " & vSheetProps(5)  
    Debug.Print "      Height          = " & vSheetProps(6)
```

End Sub

Get Name of Active Configuration Example (VBA)

This example shows how to get the name of the configuration active when the the specified SOLIDWORKS Design document was last closed.

```
' -----  
' Preconditions:  
' 1. Verify that the specified part document exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Prints to the Immediate the name of the configuration  
'    active when the the part document was last closed.  
' 2. Examine the Immediate window.  
' -----  
Option Explicit  
  
Sub main()  
    Const sFileName As String = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\fillets\knob.sldprt"  
    Dim swApp As SldWorks.SldWorks  
  
    Set swApp = Application.SldWorks  
    Debug.Print "File = " & sFileName  
    Debug.Print "  Name of active configuration = " & swApp.GetActiveConfigurationName(sFileName)  
  
End Sub
```

Get Name of Drawing Zone Example CSharp (Topic)

SOLIDWORKS API Help

Get Name of Drawing Zone Example (C#)

This example shows how to get the name of a drawing zone for the specified x and y coordinates.

```
//-----  
// Preconditions:  
// 1. Verify that public_documents\samples\tutorial\api\assem20.slddrw exists.  
// 2. Open the Immediate Window.  
//  
// Postconditions.  
// 1. Opens the specified drawing.  
// 2. Creates a new sheet named Test with four zones.  
// 3. Gets the name of the drawing zone for the specified  
//    x and y coordinates.  
// 4. Examine the Immediate window.  
//  
// NOTE: Because the drawing is used elsewhere, do not save changes.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace Macro1CSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
            DrawingDoc swDraw = default(DrawingDoc);  
            Sheet swSheet = default(Sheet);  
            ModelDoc2 swModel = default(ModelDoc2);  
            string fileName = null;  
            bool status = false;  
            int errors = 0;  
            int warnings = 0;  
            double x = 0;  
            double y = 0;  
  
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\api\\assem20.slddrw";  
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpe  
nDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);  
            swDraw = (DrawingDoc)swModel;  
  
            swSheet = (Sheet)swDraw.GetCurrentSheet();  
            swDraw.ActivateSheet(swSheet.GetName());  
  
            // Create sheet Test with four zones  
            status = swDraw.NewSheet4("Test", (int)swDwgPaperSizes_e.swDwgPaperAsize, (int)swDwgTemplates  
_e.swDwgTemplateAsize, 1, 1, true, "", 0, 0, "",  
0.5, 0.5, 0.5, 0.5, 2, 2);  
  
            swModel.ForceRebuild3(true);  
  
            swSheet = (Sheet)swDraw.GetCurrentSheet();  
            swDraw.ActivateSheet(swSheet.GetName());  
  
            // Set x and y coordinates on the sheet  
            // whose drawing zone name to get  
            x = 0.201705822198041;  
            y = 0.0242677238302502;  
  
            // Get the name of the drawing zone for x and y  
            string drawingZoneName = null;  
            drawingZoneName = swSheet.GetDrawingZone(x, y);  
            Debug.Print("Drawing zone name for specified x and y coordinates: " + drawingZoneName);  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
    }  
}
```

```
    /// </summary>  
    public SldWorks swApp;  
}
```

Get Name of Drawing Zone Example_VB (Topic)

SOLIDWORKS API Help

Get Name of Drawing Zone Example (VBA)

This example shows how to get the name of a drawing zone for the specified x and y coordinates.

```
' -----  
' Preconditions:  
' 1. Verify that public_documents\samples\tutorial\api\assem20.slddrw exists.  
' 2. Open the Immediate Window.  
'  
' Postconditions.  
' 1. Opens the specified drawing.  
' 2. Creates a new sheet named Test with four zones.  
' 3. Gets the name of the drawing zone for the specified  
'    x and y coordinates.  
' 4. Examine the Immediate window.  
'  
' NOTE: Because the drawing is used elsewhere, do not save changes.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swDraw As SldWorks.DrawingDoc  
Dim swSheet As SldWorks.Sheet  
Dim swModel As SldWorks.ModelDoc2  
Dim fileName As String  
Dim status As Boolean  
Dim errors As Long  
Dim warnings As Long  
Dim x As Double  
Dim y As Double  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\assem20.slddrw"  
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
    Set swDraw = swModel  
  
    Set swSheet = swDraw.GetCurrentSheet  
    swDraw.ActivateSheet (swSheet.GetName)  
  
    ' Create sheet Test with four zones  
    status = swDraw.NewSheet4("Test", swDwgPaperSizes_e.swDwgPaperASize,  
swDwgTemplates_e.swDwgTemplateASize, 1, 1, True, "", 0, 0, "", 0.5, 0.5, 0.5, 0.5, 2, 2)  
  
    swModel.ForceRebuild3 True  
  
    Set swSheet = swDraw.GetCurrentSheet  
    swDraw.ActivateSheet (swSheet.GetName)  
  
    ' Set x and y coordinates on the sheet  
    ' whose drawing zone name to get  
    x = 0.201705822198041  
    y = 2.42677238302502E-02  
  
    ' Get the name of the drawing zone for x and y  
    Dim drawingZoneName As String  
    drawingZoneName = swSheet.GetDrawingZone(x, y)  
    Debug.Print "Drawing zone name for specified x and y coordinates: " & drawingZoneName  
  
End Sub
```

Get Name of Drawing Zone Example (VB.NET)

This example shows how to get the name of a drawing zone for the specified x and y coordinates.

```
'-----
' Preconditions:
' 1. Verify that public_documents\samples\tutorial\api\assem20.slddrw exists.
' 2. Open the Immediate Window.
'
' Postconditions.
' 1. Opens the specified drawing.
' 2. Creates a new sheet named Test with four zones.
' 3. Gets the name of the drawing zone for the specified
'    x and y coordinates.
' 4. Examine the Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swDraw As DrawingDoc
        Dim swSheet As Sheet
        Dim swModel As ModelDoc2
        Dim fileName As String
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim x As Double
        Dim y As Double

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\assem20.slddrw"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOp
tions_Silent, "", errors, warnings)
        swDraw = swModel

        swSheet = swDraw.GetCurrentSheet
        swDraw.ActivateSheet(swSheet.GetName)

        ' Create sheet Test with four zones
        status = swDraw.NewSheet4("Test", swDwgPaperSizes_e.swDwgPaperASize, swDwgTemplates_e.swDwgTempla
teASize, 1, 1, True, "", 0, 0, "", 0.5, 0.5, 0.5, 0.5, 2, 2)

        swModel.ForceRebuild3(True)

        swSheet = swDraw.GetCurrentSheet
        swDraw.ActivateSheet(swSheet.GetName)

        ' Set x and y coordinates on the sheet
        ' whose drawing zone name to get
        x = 0.201705822198041
        y = 0.0242677238302502

        ' Get the name of the drawing zone for x and y
        Dim drawingZoneName As String
        drawingZoneName = swSheet.GetDrawingZone(x, y)
        Debug.Print("Drawing zone name for specified x and y coordinates: " & drawingZoneName)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

End Class

Get Named Entities Example (C++ COM)

This example shows how to get the named entities.

```
// -----  
// Preconditions:  
//      1) An part is open.  
//      2) At least one named entity exists in the part.  
//  
void APITestFunction()  
{  
    HRESULT          hr = S_OK;  
    VARIANT_BOOL      bRet = VARIANT_FALSE;  
    CComPtr <ISldWorks>    pSldWorks;  
    CComPtr <IModelDoc>    pModel;  
    CComQIPtr <IPartDoc>    pPart;  
    long              nEntCount = -1;  
    CComPtr <IEntity>      pEntity;  
    LPENTITY*          ppEntity = NULL;  
    long              i = -1;  
  
    // Connect to SOLIDWORKS  
    pSldWorks = TheApplication->GetSWApp();  
    ASSERT(pSldWorks);  
  
    hr = pSldWorks->get_IActiveDoc(&pModel);  
    ASSERT(pModel);  
  
    pPart = pModel;  
    ASSERT(pPart);  
  
    hr = pPart->GetNamedEntitiesCount (&nEntCount);  
    ASSERT(nEntCount > 0);  
  
    ppEntity = new LPENTITY[nEntCount];  
    ASSERT(ppEntity);  
  
    ZeroMemory(ppEntity, nEntCount * sizeof(LPENTITY));  
  
    hr = pPart->IGetNamedEntities(nEntCount, ppEntity);  
    ASSERT(S_OK == hr);  
  
    for (i = 0; i < nEntCount; i++)  
    {
```

```
CComPtr <IEntity>      pEntity;  
CComBSTR               sEntName;
```

```
pEntity.Attach(ppEntity[i]);
```

```
hr = pPart->IGetEntityName(pEntity, &sEntName);
```

```
OutputDebugString(sEntName);
```

```
OutputDebugString(_T("\n"));
```

```
}
```

```
delete [] ppEntity;
```

```
} //void APITestFunction()
```

```
// -----
```

Get Named Entities Example (VBA)

This example shows how to get named entities.

```
' -----  
' Preconditions:  
' 1. Part document with one or more named entities  
'    (faces, surfaces, bodies, or edges) is open.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the number of named entities and their  
'    names.  
' 2. Examine the Immediate window.  
' -----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swPart As SldWorks.PartDoc  
    Dim vNamedEntity As Variant  
    Dim swEnt As Entity  
    Dim nEntCount As Long  
    Dim i As Long  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swPart = swModel  
  
    ' Get the full path name for the part  
    Debug.Print "File = " & swModel.GetPathName  
  
    ' Determine the total number of named entities  
    ' in the part and print that value  
    Debug.Print "  Number named entities = " & swPart.GetNamedEntitiesCount  
    If swPart.GetNamedEntitiesCount = 0 Then  
        Debug.Print "  No named entities."  
        Exit Sub  
    End If  
    Debug.Print ""  
    vNamedEntity = swPart.GetNamedEntities  
    For i = 0 To UBound(vNamedEntity)  
        Set swEnt = vNamedEntity(i)  
        ' Print the name of each named entity  
        Debug.Print "  Entity(" & i & ") = " & swPart.GetEntityName(swEnt)  
    Next i  
  
End Sub
```

Get Names and Show Model Break Views Example (C#)

This example shows how to:

- get the names of the Model Break Views in the current configuration of an active model.
- show one of the Model Break Views.

```
//-----  
// Preconditions:  
// 1. Open a part document with multiple  
//    hidden Model Break Views.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Gets the number and names of the Model Break  
//    Views in the current configuration of the  
//    active model.  
// 2. Shows one of the Model Break Views.  
// 3. Examine the Immediate window and the  
//    graphics area.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace ModelBreakViewsCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel;  
            ModelDocExtension swModelDocExt;  
            object[] names = null;  
            object oNames = null;  
            int nbr;  
            int i;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swModelDocExt = (ModelDocExtension)swModel.Extension;  
  
            //Get and print number and names of Model Break  
            //Views in current configuration of the active  
            //model  
            nbr = swModelDocExt.GetModelBreakViewNames(out oNames);  
            names = (object[])oNames;  
            Debug.Print("Model Break Views:");  
            Debug.Print("  Number: " + nbr);  
            Debug.Print("  Names: ");  
            for (i = 0; i <= nbr - 1; i++)  
            {  
                Debug.Print("    " + names[i]);  
            }  
  
            //Show Model Break View  
            string name;  
            name = (string)names[nbr - 1];  
            swModelDocExt.ShowModelBreakView(true, name);  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
        public SldWorks swApp;  
    }  
}
```

Get Names and Show Model Break Views Example (VBA)

This example shows how to:

- get the names of the Model Break Views in the current configuration of an active model.
- show one of the Model Break Views.

```
'-----  
' Preconditions:  
' 1. Open a part document with multiple  
'    hidden Model Break Views.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the number and names of the Model Break  
'    Views in the current configuration of the  
'    active model.  
' 2. Shows one of the Model Break Views.  
' 3. Examine the Immediate window and the  
'    graphics area.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim names As Variant  
Dim name As String  
Dim nbr As Long  
Dim i As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
  
    'Get and print number and names of Model Break  
    'Views in current configuration of the active  
    'model  
    nbr = swModelDocExt.GetModelBreakViewNames(names)  
    ReDim Preserve names(nbr - 1)  
    Debug.Print "Model Break Views:"  
    Debug.Print "  Number: " & nbr  
    Debug.Print "  Names: "  
    For i = 0 To nbr - 1  
        name = CStr(names(i))  
        Debug.Print "    " & name  
    Next  
  
    'Show Model Break View  
    swModelDocExt.ShowModelBreakView True, name  
  
End Sub
```

Get Names and Show Model Break Views Example (VB.NET)

This example shows how to:

- get the names of the Model Break Views in the current configuration of an active model.
- show one of the Model Break Views.

```
'-----  
' Preconditions:  
' 1. Open a part document with multiple  
'    hidden Model Break Views.  
' 2. Open the Immediate window.  
'-----  
' Postconditions:  
' 1. Gets the number and names of the Model Break  
'    Views in the current configuration of the  
'    active model.  
' 2. Shows one of the Model Break Views.  
' 3. Examine the Immediate window and the  
'    graphics area.  
'-----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Dim swModel As ModelDoc2  
    Dim swModelDocExt As ModelDocExtension  
    Dim names As Object  
    Dim nbr As Integer  
    Dim i As Integer  
  
    Public Sub Main()  
  
        swModel = swApp.ActiveDoc  
        swModelDocExt = swModel.Extension  
  
        'Get and print number and names of Model Break  
        'Views in current configuration of the active  
        'model  
        nbr = swModelDocExt.GetModelBreakViewNames(names)  
  
        Debug.Print("Model Break Views:")  
        Debug.Print("  Number: " & nbr)  
        Debug.Print("  Names: ")  
        For i = 0 To (nbr - 1)  
            Debug.Print("    " & names(i))  
        Next  
        'Show Model Break View  
        swModelDocExt.ShowModelBreakView(True, names(nbr - 1))  
  
    End Sub  
  
    ''' <summary>  
    ''' The SldWorks swApp variable is pre-assigned for you.  
    ''' </summary>  
    Public swApp As SldWorks  
  
End Class
```

Get Names of Annotations Example CSharp (Topic)

SOLIDWORKS API Help

Get Names of Annotations Example (C#)

This example shows show to:

- get the names of all of the annotations in a drawing.
- select all of the annotations in a drawing.

```
//-----  
// Preconditions:  
// 1. Verify that the specified drawing document to open exists.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Opens the specified drawing document.  
// 2. Iterates the drawing views and gets and selects  
//    each annotation in each drawing view.  
// 3. Examine the Immediate window and drawing.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace GetNamesCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            DrawingDoc swDrawing = default(DrawingDoc);  
            View swDrView = default(View);  
            object[] annArray = null;  
            object obj = null;  
            Annotation currAnn = default(Annotation);  
            string fileName = null;  
            int errors = 0;  
            int warnings = 0;  
  
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\api\\cylinder20.SLDDRW";  
            swDrawing = (DrawingDoc)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)sw  
OpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);  
  
            //Get drawing views and names of annotations in  
            //each drawing view  
            swDrView = (View)swDrawing.GetFirstView();  
            //First drawing view is the sheet, so get next drawing view  
            swDrView = (View)swDrView.GetNextView();  
            while ((swDrView != null))  
            {  
                Debug.Print("Name of drawing view: " + swDrView.GetName2());  
                annArray = (object[])swDrView.GetAnnotations();  
                if ((annArray != null))  
                {  
                    foreach (object obj_loopVariable in annArray)  
                    {  
                        obj = (object)obj_loopVariable;  
                        currAnn = (Annotation)obj;  
                        Debug.Print("  Name of annotation: " + currAnn.GetName());  
                        currAnn.Select3(true, null);  
                    }  
                }  
                swDrView = (View)swDrView.GetNextView();  
            }  
        }  
    }  
  
    /// <summary>  
    /// The SldWorks swApp variable is pre-assigned for you.  
    /// </summary>  
    public SldWorks swApp;
```

} }

Get Names of Annotations Example (VBA)

This example shows show to:

- get the names of all of the annotations in a drawing.
- select all of the annotations in a drawing.

```
'-----  
' Preconditions:  
' 1. Verify that the specified drawing document to open exists.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified drawing document.  
' 2. Iterates the drawing views and gets and selects  
'    each annotation in each drawing view.  
' 3. Examine the Immediate window and drawing.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swDrawing As SldWorks.DrawingDoc  
Dim swDrView As SldWorks.View  
Dim annArray As Variant  
Dim obj As Variant  
Dim currAnn As SldWorks.Annotation  
Dim fileName As String  
Dim errors As Long  
Dim warnings As Long  
  
Sub main()  
    Set swApp = Application.SldWorks  
  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\api\cylinder20.SLDDRW"  
    Set swDrawing = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING,  
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)  
  
    'Get drawing views and names of annotations in  
    'each drawing view  
    Set swDrView = swDrawing.GetFirstView  
    'First drawing view is the sheet, so get first drawing view  
    Set swDrView = swDrView.GetNextView  
  
    While Not swDrView Is Nothing  
        Debug.Print "Name of drawing view: " & swDrView.GetName2  
        annArray = swDrView.GetAnnotations  
        If Not IsEmpty(annArray) Then  
            For Each obj In annArray  
                Set currAnn = obj  
                Debug.Print "  Name of annotation: " & currAnn.GetName  
                currAnn.Select3 True, Nothing  
            Next obj  
        End If  
        Set swDrView = swDrView.GetNextView  
    Wend  
  
End Sub
```

Get Names of Annotations Example (VB.NET)

This example shows show to:

- get the names of all of the annotations in a drawing.
- select all of the annotations in a drawing.

```
'-----
' Preconditions:
' 1. Verify that the specified drawing document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing document.
' 2. Iterates the drawing views and gets and selects
'    each annotation in each drawing view.
' 3. Examine the Immediate window and drawing.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swDrawing As DrawingDoc
        Dim swDrView As View
        Dim annArray() As Object
        Dim obj As Object
        Dim currAnn As Annotation
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\cylinder20.SLDDRW"
        swDrawing = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDoc
Options_Silent, "", errors, warnings)

        'Get drawing views and names of annotations in
        'each drawing view
        swDrView = swDrawing.GetFirstView
        'First drawing view is the sheet, so get next drawing view
        swDrView = swDrView.GetNextView
        While Not swDrView Is Nothing
            Debug.Print("Name of drawing view: " & swDrView.GetName2)
            annArray = swDrView.GetAnnotations
            If Not annArray Is Nothing Then
                For Each obj In annArray
                    currAnn = obj
                    Debug.Print(" Name of annotation: " & currAnn.GetName)
                    currAnn.Select3(True, Nothing)
                Next obj
            End If
            swDrView = swDrView.GetNextView
        End While

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class
```

Get Names of Bodies in Multibody Part Example (VBA)

This example shows how to get the names of the bodies in each multibody part in an assembly.

```
' -----
' Preconditions:
' 1. Verify that the specified files exist:
'   * assembly document template
'   * both multibody part documents
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens both multibody parts.
' 2. Opens a new assembly document.
' 3. Adds the open multibody parts as components
'   to the assembly.
' 4. Traverses the components in the assembly and gets
'   and prints the names of the bodies in each
'   component.
' 5. Examine the Immediate window.
'
' NOTE: Because the multibody parts are used
' elsewhere, do not save changes.
' -----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swAssembly As SldWorks.AssemblyDoc
Dim swComponent1 As SldWorks.Component2
Dim swComponent2 As SldWorks.Component2
Dim swPart As SldWorks.PartDoc
Dim multibodyPart1 As String
Dim multibodyPart2 As String
Dim errors As Long
Dim warnings As Long
Dim vBody As Variant
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open the multibody parts, new assembly document, and add
    'the multibody parts as components to the assembly
    multibodyPart1 = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\multibody\multi_bridge.sldprt"
    multibodyPart2 = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\multibody\multi_inter.sldprt"
    Set swModel = swApp.OpenDoc6(multibodyPart1, swDocumentTypes_e.swDocPART,
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModel = swApp.OpenDoc6(multibodyPart2, swDocumentTypes_e.swDocPART,
swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks
2014\templates\assembly.asmdot", swDwgPaperSizes_e.swDwgPaperA0size, 0, 0)
    Set swAssembly = swModel
    Set swComponent1 = swAssembly.AddComponent5(multibodyPart1,
swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfig, "", False, "",
-1.11307660915827E-04, 5.87355354468855E-05, -1)
    Set swComponent2 = swAssembly.AddComponent5(multibodyPart2,
swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfig, "", False, "",
-0.218915538687725, 0.115372315514833, 4.34772581793368E-02)
    swModel.ViewZoomtofit2
```

```

Debug.Print "File = " & swModel.GetPathName

'Process assembly
Select Case swModel.GetType
    Case swDocPART
        Set swPart = swModel
        'Solid bodies
        vBody = swPart.GetBodies2(swSolidBody, True)
        SelectBodies swApp, swModel, vBody, ""
        'Surface bodies
        vBody = swPart.GetBodies2(swSheetBody, True)
        SelectBodies swApp, swModel, vBody, ""
        'Assembly
    Case swDocASSEMBLY
        ProcessAssembly swApp, swModel
    Case Else
        Exit Sub
End Select

End Sub

Sub SelectBodies(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, vBody As Variant, sPadStr As String)

    Dim swModExt As SldWorks.ModelDocExtension
    Dim swBody As SldWorks.Body2
    Dim sBodySelStr As String
    Dim sBodyTypeSelStr As String
    Dim i As Long
    Dim status As Boolean

    If IsEmpty(vBody) Then Exit Sub
    Set swModExt = swModel.Extension

    For i = 0 To UBound(vBody)
        Set swBody = vBody(i)
        sBodySelStr = swBody.GetSelectionId
        Debug.Print " " & sPadStr & sBodySelStr
        Select Case swBody.GetType
            Case swSolidBody
                sBodyTypeSelStr = "SOLIDBODY"
            Case swSheetBody
                sBodyTypeSelStr = "SURFACEBODY"
            Case Else
                Debug.Assert False
        End Select
        status = swModExt.SelectByID2(sBodySelStr, sBodyTypeSelStr, 0#, 0#, 0#, True, 0, Nothing, swSelectOptionDefault)
    Next i

End Sub

Sub ProcessAssembly(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2)

    Dim swConfigMgr As SldWorks.ConfigurationManager
    Dim swConf As SldWorks.Configuration
    Dim swRootComp As SldWorks.Component2

    Set swConfigMgr = swModel.ConfigurationManager
    Set swConf = swConfigMgr.ActiveConfiguration
    Set swRootComp = swConf.GetRootComponent
    ProcessComponent swApp, swModel, swRootComp, 1

End Sub

Sub ProcessComponent(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swComp As SldWorks.Component2, nLevel As Long)

    Dim vChildComp As Variant
    Dim swChildComp As SldWorks.Component2
    Dim sPadStr As String
    Dim vBody As Variant
    Dim i As Long

```

```

For i = 0 To nLevel - 1
    sPadStr = sPadStr + "  "
Next i

Debug.Print sPadStr & swComp.Name2 & " <" & swComp.ReferencedConfiguration & ">"

'Solid bodies
vBody = swComp.GetBodies2(swSolidBody)
SelectBodies swApp, swModel, vBody, sPadStr

'Surface bodies
vBody = swComp.GetBodies2(swSheetBody)
SelectBodies swApp, swModel, vBody, sPadStr

'Components
vChildComp = swComp.GetChildren
For i = 0 To UBound(vChildComp)
    Set swChildComp = vChildComp(i)
    ProcessComponent swApp, swModel, swChildComp, nLevel + 1
Next i

```

End Sub

Get Names of Components and Window Handle, and DIBSECTION Example CSharp

(Topic)

SOLIDWORKS API Help

Get Names of Components, Window Handles, and DIBSECTIONs Example (C#)

This example shows how to get:

- names of the assembly components in an open, inactive, assembly document.
- window handles and pointers to images of model documents.

```
//-----  
// Preconditions:  
// 1. Verify that the specified model documents to open exist.  
//  
// Postconditions:  
// 1. Opens the specified model documents.  
// 2. Prints to the Immediate window:  
//    * paths and file names of:  
//    * assembly components of the open, inactive, assembly document  
//    * open, active, part document  
//    * open, inactive, assembly document  
//    * whether the assembly components, part, and assembly documents  
//    are visible  
// 3. Prints to the Immediate window each visible document's:  
//    * Microsoft window handle  
//    * pointer to its image as it looks in Normal view  
//    or in the print preview  
// 4. Examine the Immediate window.  
//  
// NOTE: Because the models are used elsewhere, do not save changes.  
//-----  
  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace GetViewHwndCSharp.csproj  
{  
    public partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            ModelView swModView = default(ModelView);  
            string fileName = null;  
            int errors = 0;  
            int warnings = 0;  
  
            //Open assembly document, then open part document  
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\advdrawings\\bowl and chute.sldasm";  
            swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpen  
DocOptions_Silent, "", ref errors, ref warnings);  
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\advdrawings\\switch.sldprt";  
            swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocO  
ptions_Silent, "", ref errors, ref warnings);  
  
            //NOTE: Assembly components are open, but they  
            //are not visible until opened by the user.  
  
            //Get first open document  
            swModel = (ModelDoc2)swApp.GetFirstDocument();  
  
            while ((swModel != null))  
            {  
                Debug.Print(swModel.GetPathName());  
                //Get whether assembly component, part, or assembly is visible  
                Debug.Print("    Visible? " + swModel.Visible);  
                swModView = (ModelView)swModel.ActiveView;  
                while ((swModView != null))
```

```

    {
        //NOTE: Model views exist only for visible model documents.

        //Get Microsoft window handle and pointer to image
        //for each visible model document
        //64-bit SOLIDWORKS
        Debug.Print("    Microsoft window handle = " + swModView.GetViewHwndx64());
        Debug.Print("    Pointer to image = " + swModView.GetViewDIBx64());

        swModView = (ModelView)swModView.GetNext();
    }
    //Get next open document
    swModel = (ModelDoc2)swModel.GetNext();
}
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Get Names of Components and Window Handle, and DIBSECTION Example_VB

(Topic)

SOLIDWORKS API Help

Get Names of Components, Window Handles, and DIBSECTIONs Example (VBA)

This example shows how to get:

- names of the assembly components in an open, inactive, assembly document.
- window handles and pointers to images of model documents.

```
'-----  
' Preconditions:  
' 1. Verify that the specified model documents to open exist.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Opens the specified model documents.  
' 2. Prints to the Immediate window:  
'    * paths and file names of:  
'      * assembly components of the open, inactive, assembly document  
'      * open, active, part document  
'      * open, inactive, assembly document  
'    * whether the assembly components, part, and assembly documents  
'      are visible  
' 3. Prints to the Immediate window each visible document's:  
'    * Microsoft window handle  
'    * pointer to its image as it looks in Normal view  
'      or in the print preview  
' 4. Examine the Immediate window.  
'  
' NOTE: Because the models are used elsewhere, do not save changes.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModView As SldWorks.ModelView  
    Dim fileName As String  
    Dim errors As Long  
    Dim warnings As Long  
  
    Set swApp = Application.SldWorks  
  
    'Open assembly document, then open part document  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\ Bowl  
and chute.sldasm"  
    swApp.OpenDoc6 fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent,  
    "", errors, warnings  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS  
2018\samples\tutorial\advdrawings\switch.sldprt"  
    swApp.OpenDoc6 fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "",  
    errors, warnings  
  
    'NOTE: Assembly components are open, but they  
    'are not visible until opened by the user.  
  
    'Get first open document  
    Set swModel = swApp.GetFirstDocument  
  
    Do While Not swModel Is Nothing  
        Debug.Print swModel.GetPathName  
        'Get whether assembly component, part, or assembly is visible  
        Debug.Print "    Visible? " & swModel.Visible  
        Set swModView = swModel.ActiveView
```

```

Do While Not swModView Is Nothing

    'NOTE: Model views exist only for visible model documents.

    'Get Microsoft window handle and pointer to image
    'for each visible model document
    '64-bit SOLIDWORKS
    Debug.Print "    Microsoft window handle = " & swModView.GetViewHwndx64
    Debug.Print "    Pointer to image = " & swModView.GetViewDIBx64

    Set swModView = swModView.GetNext
Loop
'Get next open document
Set swModel = swModel.GetNext
Loop
End Sub

```

Get Names of Components and Window Handle, and DIBSECTION Example_VBNET

(Topic)

SOLIDWORKS API Help

Get Names of Components, Window Handles, and DIBSECTIONS Example (VB.NET)

This example shows how to get:

- names of the assembly components in an open, inactive, assembly document.
- window handles and pointers to images of model documents.

```
'-----
' Preconditions:
' 1. Verify that the specified model documents to open exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified model documents.
' 2. Prints to the Immediate window:
'   * paths and file names of:
'     * assembly components of the open, inactive, assembly document
'     * open, active, part document
'     * open, inactive, assembly document
'   * whether the assembly components, part, and assembly documents
'     are visible
' 3. Prints to the Immediate window each visible document's:
'   * Microsoft window handle
'   * pointer to its image as it looks in Normal view
'     or in the print preview
' 4. Examine the Immediate window.
'
' NOTE: Because the models are used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModView As ModelView
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer

        'Open assembly document, then open part document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\owl and chute.sldasm"
        swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Sil
ent, "", errors, warnings)
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\switch.sldprt"
        swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent,
"", errors, warnings)

        'NOTE: Assembly components are open, but they
        'are not visible until opened by the user.

        'Get first open document
        swModel = swApp.GetFirstDocument

        Do While Not swModel Is Nothing
            Debug.Print(swModel.GetPathName)
            'Get whether assembly component, part, or assembly is visible
            Debug.Print("    Visible? " & swModel.Visible)
            swModView = swModel.ActiveView
            Do While Not swModView Is Nothing

                'NOTE: Model views exist only for visible model documents.

                'Get Microsoft window handle and pointer to image
```

```
        'for each visible model document
        '64-bit SOLIDWORKS
        Debug.Print("    Microsoft window handle = " & swModView.GetViewHwndx64)
        Debug.Print("    Pointer to image = " & swModView.GetViewDIBx64)

        swModView = swModView.GetNext
    Loop
    'Get next open document
    swModel = swModel.GetNext
Loop
```

End Sub

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

End Class

Get Names of Creators of Features Example (C#)

This example shows how to get the names of the creators of features in a part document.

```
//-----  
// Preconditions:  
// 1. Verify that the specified part exists.  
// 2. Open the Immediate window.  
//  
// Postconditions:  
// 1. Gets the names of the features and their creators.  
// 2. Examine the Immediate window.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace CloseDocCSharp.csproj  
{  
  
    partial class SolidWorksMacro  
    {  
  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            Feature swFeat = default(Feature);  
            string Filename = null;  
            int errors = 0;  
            int warnings = 0;  
  
            Filename = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS  
2018\\samples\\tutorial\\api\\cstick.sldprt";  
  
            // Open document  
            swApp.OpenDoc6(Filename, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocO  
ptions_Silent, "", ref errors, ref warnings);  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
  
            // Get first feature in this part document  
            swFeat = (Feature)swModel.FirstFeature();  
  
            // Iterate over features in this part document in  
            // FeatureManager design tree  
            while ((swFeat != null))  
            {  
                // Write the name of the feature and its  
                // creator to the Immediate window  
                Debug.Print(" Feature " + swFeat.Name + " created by " + swFeat.CreatedBy);  
  
                // Get next feature in this part document  
                swFeat = (Feature)swFeat.GetNextFeature();  
            }  
  
            swApp.CloseDoc(Filename);  
        }  
  
        /// <summary>  
        /// The SldWorks swApp variable is pre-assigned for you.  
        /// </summary>  
  
        public SldWorks swApp;  
    }  
}
```

Get Names of Creators of Features Example (VBA)

This example shows how to get the names of the creators of the features in multiple part documents.

```
' -----  
' Preconditions: Create the input file c:\temp\SOLIDWORKSFileNames.txt  
' in a text editor and add the paths and file names of the  
' part documents for which you want the creators of the  
' features identified.  
'  
' Postconditions:  
' 1. Opens c:\temp\SOLIDWORKSFileNames.txt.  
' 2. Creates and opens c:\temp\SOLIDWORKSFileNamesFeatureCreators.txt.  
' 3. Reads the name of the first part document.  
' 4. Writes the name of the part document to  
'    c:\temp\SOLIDWORKSFileNamesFeatureCreators.txt.  
' 5. Opens the part document and iterates over its FeatureManager design tree  
'    and writes the name of each feature and its creator to  
'    c:\temp\SOLIDWORKSFileNamesFeatureCreators.txt.  
' 6. Closes the part document.  
' 7. Repeats steps 2 - 5 for each subsequent part document in  
'    c:\temp\SOLIDWORKSFileNamesFeatureCreators.txt.  
' 8. Closes c:\temp\SOLIDWORKSFileNamesFeatureCreators.txt and  
'    c:\temp\SOLIDWORKSFileNamesFeatureCreators.txt.  
' 9. Open c:\temp\SOLIDWORKSFileNamesFeatureCreators.txt in a text editor  
'    and examine the contents of the file.  
' -----
```

Option Explicit

Sub main()

```
Dim Filename As String  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swFeat As SldWorks.Feature  
Dim fileerror As Long  
Dim filewarning As Long  
  
' Disable Visual Basic error on implicit Query Interface  
On Error Resume Next  
  
Set swApp = Application.SldWorks  
  
' Open the input file with the list of part documents to  
' open and get feature and name of its creator  
Open "C:\temp\SOLIDWORKSFileNames.txt" For Input As #1  
  
' Create and open the output file where to write the  
' name of the part document and its features and their creators  
Open "C:\temp\SOLIDWORKSFileNamesFeatureCreators.txt" For Output As #2  
    ' While not at the end of the input file, read the name of  
    ' the part document to open  
    Do While Not EOF(1)  
        Line Input #1, Filename  
        'Write the name of the part document to open to the output file  
        Write #2, "===== "  
        Write #2, Filename  
  
        ' Open document  
        swApp.OpenDoc6 Filename, swDocPART, swOpenDocOptions_Silent, "", fileerror, filewarning  
        Set swModel = swApp.ActiveDoc  
  
        ' Get first feature in this part document  
        Set swFeat = swModel.FirstFeature  
        ' Iterate over features in this part document in FeatureManager design tree
```

```

        Do While Not swFeat Is Nothing
            ' Write the name of the feature and its creator to the output file
            Write #2, "  Feature " & swFeat.Name & " created by " & swFeat.CreatedBy
            ' Get next feature in this part document
            Set swFeat = swFeat.GetNextFeature
        Loop
        ' Close part document
        swApp.CloseDoc Filename
    Loop

    ' Close the input file
    Close #1

    'Close the output file
    Write #2, "=====
    Close #2

End Sub

```

Get Names of Creators of Features Example_VBNET (Topic)

SOLIDWORKS API Help

Get Names of Creators of Features Example (VB.NET)

This example shows how to get the names of the creators of features in a part document.

```
' -----
' Preconditions:
' 1. Verify that the specified part exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the names feature and their creators.
' 2. Examine the Immediate window.
' -----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swFeat As Feature
        Dim Filename As String
        Dim errors As Integer
        Dim warnings As Integer

        Filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\cstick.sldprt"

        ' Open document
        swApp.OpenDoc6(Filename, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent,
"", errors, warnings)
        swModel = swApp.ActiveDoc

        ' Get first feature in this part document
        swFeat = swModel.FirstFeature

        ' Iterate over features in this part document in
        ' FeatureManager design tree

        Do While Not swFeat Is Nothing
            ' Write the name of the feature and its
            ' creator to the Immediate window
            Debug.Print("  Feature " & swFeat.Name & " created by " & swFeat.CreatedBy)

            ' Get next feature in this part document
            swFeat = swFeat.GetNextFeature
        Loop

        swApp.CloseDoc(Filename)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Get Names of Creators of Features Example (C++ COM)

This example shows how to get the names of the creators of the features in multiple part documents.

- [FeatureCreatedBy.cpp](#)
- [stdafx.h](#)
- [stdafx.cpp](#)

FeatureCreatedBy.cpp

```
/*
PRECONDITIONS:
* SOLIDWORKS Design is open and the specified input file exists in this folder:
  c:\SOLIDWORKS\Filenames.txt.
* The part documents listed in the input file and used by
  this application were installed when SOLIDWORKS was installed:
  - C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\toolbox\braceleft.sldprt
  - C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\toolbox\braceright.sldprt
  - C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\toolbox\baseplate.sldprt

POSTCONDITIONS:
* The specified output file is created in the specified folder:
  c:\SOLIDWORKS\Filenames\FeatureCreators.txt.
* The names of the part documents and their features
  and the names of the creators of those features.
*/

#include "stdafx.h"
#include <fstream> //Stream class to both read from and write to files
#include <string>

CComBSTR makeBstr(string input); //makeBstr function prototype

using namespace std;

int _tmain()
{
    ifstream infile;
    ofstream outfile;

    //Initialize COM
    CoInitialize(NULL);

    //Use ATL smart pointers
    CComPtr<ISldWorks> swApp;
    CComPtr<IModelDoc2> swModel;
    CComPtr<IFeature> swFeat;
    CComPtr<IFeature> swNextFeat;
```

```
long lErrors;
long lWarnings;

string sSOLIDWORKSFilename;

CComBSTR sDefaultConfiguration(L"Default");
CComBSTR sFeatureName(L "");
CComBSTR sFeatureCreator(L "");

HRESULT hres = NOERROR;

//Open input file
infile.open("C:\\SOLIDWORKS\\FileNames.txt", std::ios_base::in);
//Make sure input file exists
if(!infile)
{
    cout << "ERROR: Cannot open the input file." << endl;
    return 1;
}

//Open output file
outfile.open("c:\\SOLIDWORKS\\FileNames\\FeatureCreators.txt", std::ios_base::out );
//Make sure output file exists
if( !outfile)
{
    cout << "ERROR: Cannot open the output file." << endl;
    infile.close();
    return 1;
}

outfile << "===== " << endl;

//Connect to currently running instance of SOLIDWORKS Design
hres = swApp.CoCreateInstance(__uuidof(SldWorks), NULL, CLSCTX_LOCAL_SERVER);

//Check results
if (S_OK != hres)
    return 1;

while (!infile.eof())
{
    //Read part document name from SOLIDWORKS\\FileNames.txt
    getline (infile, sSOLIDWORKSFilename);

    //Convert string to CComBSTR by calling function makeBstr
    CComBSTR sSWFilename = (makeBstr(sSOLIDWORKSFilename));

    cout << sSOLIDWORKSFilename << endl;
```

```

//Open part document
hres = swApp->OpenDoc6(sSWFilename, swDocPART, swOpenDocOptions_Silent, sDefaultConfiguration, &lErrors, &lWarnings,
&swModel);

//Check results
if (S_OK != hres || swModel == NULL)
    return 1;

//Write name of part document in SOLIDWORKSFileNamesFeatureCreators.txt
outfile << sSOLIDWORKSFilename << endl;
outfile << "===== " << endl;

//Get first feature
swModel->IFirstFeature(&swFeat);

//Iterate over features in this part document in the FeatureManager design tree
while(swFeat)
{
    //Get feature name
    swFeat->get_Name(&sFeatureName);

    //Get creator of feature
    swFeat->get_CreatedBy(&sFeatureCreator);

    //Convert feature name and creator of feature to
    //string and write to output file
    CW2CT sFeatureNameString(sFeatureName);
    CW2CT sFeatureCreatorString(sFeatureCreator);

    string sFNS = _bstr_t(sFeatureNameString);
    string sFCS = _bstr_t(sFeatureCreatorString);
    outfile <<
"Feature " << sFNS << " created by " << sFCS << endl;
    //Get next feature in this part document
    swFeat->IGetNextFeature(&swNextFeat);

    swFeat.Release();
    swFeat = swNextFeat;
    swNextFeat.Release();
}

outfile << "===== " << endl;

//Close the part document
swApp->CloseDoc(sSWFilename);

swModel.Release();
swFeat.Release();
}

```

```

        //Close input and output files
        infile.close();
        outfile.close();

        swApp.Release();

        //Uninitialize COM
        CoUninitialize();

        return 0;
    }

//Function to convert string to CComBSTR
CComBSTR makeBstr(string input)
{
    char* CStyleString = (char*) input.c_str();
    CComBSTR retval(CStyleString);
    return retval;
}

```

[Back to top](#)

stdafx.h

```

/// stdafx.h : include file for standard system include files,
// or project-specific include files that are used frequently, but
// are changed infrequently
//

#pragma once

#define WIN32_LEAN_AND_MEAN // Exclude rarely-used stuff from Windows headers
#include <stdio.h>
#include <tchar.h>
#define _ATL_CSTRING_EXPLICIT_CONSTRUCTORS // some CString constructors will be explicit

#include <windows.h>
#include <atlbase.h>
#include <iostream>

using namespace std; //We will use the standard C++ libraries for text output.

#import "sldworks.tlb" raw_interfaces_only, raw_native_types, no_namespace, named_guids //the SOLIDWORKS type library
#import "swconst.tlb" raw_interfaces_only, raw_native_types, no_namespace, named_guids //the SOLIDWORKS constant type library

```

[Back to top](#)

stdafx.cpp

```

// stdafx.cpp : source file that includes just the standard includes
// FeatureCreatedBy.pch will be the pre-compiled header
// stdafx.obj will contain the pre-compiled type information
#include "stdafx.h"

```

[Back to top](#)

Get Names of Open Documents Example (VBA)

This example shows how to get the names of the open documents, the names of the referenced model documents, and their visibility.

```
' -----  
' Preconditions:  
' 1. Open multiple model documents.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the names of the open and referenced model documents.  
' 2. Examine the Immediate window.  
' -----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModelTitle As SldWorks.ModelDoc2  
    Dim swModelPath As SldWorks.ModelDoc2  
    Dim cTitle As New Collection  
    Dim vTitle As Variant  
    Dim cPath As New Collection  
    Dim vPath As Variant  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.GetFirstDocument  
  
    While Not swModel Is Nothing  
        ' Assembly components are opened, but are not visible  
        ' until opened by the user  
        Debug.Print swModel.GetPathName & " [" & swModel.Visible & "]"  
        Debug.Print " " & swModel.GetTitle & " [" & swModel.GetType & "]"  
  
        ' The document name contains a filename extension  
        ' if the document has been saved  
        ' and is subject to Microsoft File Explorer setting;  
        ' the document name does not contain a  
        ' filename extension if the document has not been saved;  
        ' ModelDoc2::GetPathName is blank until the file is saved  
        cTitle.Add swModel.GetTitle  
        cPath.Add swModel.GetPathName  
        Set swModel = swModel.GetNext  
    Wend  
  
    Debug.Print " "  
  
    For Each vTitle In cTitle  
        Set swModelTitle = swApp.GetOpenDocument(vTitle): Debug.Assert Not swModelTitle Is Nothing  
        Debug.Print swModelTitle.GetPathName & " [" & swModelTitle.Visible & "]"  
        Debug.Print " " & swModelTitle.GetTitle & " [" & swModelTitle.GetType & "]"  
    Next vTitle  
  
    Debug.Print " "  
  
    For Each vPath In cPath  
        Set swModelPath = swApp.GetOpenDocument(vPath)  
        If Not swModelPath Is Nothing Then  
            Debug.Print swModelPath.GetPathName & " [" & swModelPath.Visible & "]"  
            Debug.Print " " & swModelPath.GetTitle & " [" & swModelPath.GetType & "]"  
        End If  
    Next vPath
```

End Sub

Get Names of Sketch Segments Example (C#)

This example shows how to get the names of the sketch segments in a sheet metal bend.

```
//-----  
// Preconditions:  
// 1. Open a sheet metal part or a drawing containing a bend.  
// 2. Unsuppress the flat pattern for the bend.  
// 3. Select a bend feature in the FeatureManager design tree.  
// 4. Ensure that the namespace matches the name of your C# project.  
//  
// Postconditions: Inspect the Immediate window for information about each  
// sketch segment in the bend feature.  
//-----  
using Microsoft.VisualBasic;  
using System;  
using System.Collections;  
using System.Collections.Generic;  
using System.Data;  
using System.Diagnostics;  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
namespace FlatPatternSketchSegments_CSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            MathUtility swMathUtil = default(MathUtility);  
            ModelDoc2 swModel = default(ModelDoc2);  
            SelectionMgr swSelMgr = default(SelectionMgr);  
            Feature swFeat = default(Feature);  
            OneBendFeatureData swOneBend = default(OneBendFeatureData);  
            Object[] vSketchSegs = null;  
            SketchSegment swSketchSeg = default(SketchSegment);  
            Sketch swSketch = default(Sketch);  
            Feature swSketchFeat = default(Feature);  
            SketchLine swSketchLine = default(SketchLine);  
            SketchPoint swSkStartPt = default(SketchPoint);  
            SketchPoint swSkEndPt = default(SketchPoint);  
            SelectData swSelData = default(SelectData);  
            double[] nPt = new double[3];  
            MathPoint swStartPt = default(MathPoint);  
            MathPoint swEndPt = default(MathPoint);  
            MathTransform swSkXform = default(MathTransform);  
            int[] vID = null;  
            int i = 0;  
  
            swMathUtil = (MathUtility)swApp.GetMathUtility();  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swSelMgr = (SelectionMgr)swModel.SelectionManager;  
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);  
            swSelData = swSelMgr.CreateSelectData();  
            swOneBend = (OneBendFeatureData)swFeat.GetDefinition();  
            Debug.Print("Type of bend (swBendType_e): " + swOneBend.GetType());  
            Debug.Print("Number of sketch segments: " +  
swOneBend.GetFlatPatternSketchSegmentCount2());  
            vSketchSegs = (Object[])swOneBend.FlatPatternSketchSegments2;
```

```

for (i = 0; i <= vSketchSegs.GetUpperBound(0); i++)
{
    swSketchSeg = (SketchSegment)vSketchSegs[i];
    swSketch = swSketchSeg.GetSketch();
    swSketchLine = (SketchLine)swSketchSeg;
    swSkStartPt = (SketchPoint)swSketchLine.GetStartPoint2();
    swSkEndPt = (SketchPoint)swSketchLine.GetEndPoint2();
    vID = (int[])swSketchSeg.GetID();

    // Get sketch feature
    swSketchFeat = (Feature)swSketch;
    swSkXform = swSketch.ModelToSketchTransform;
    swSkXform = (MathTransform)swSkXform.Inverse();

    nPt[0] = swSkStartPt.X;
    nPt[1] = swSkStartPt.Y;
    nPt[2] = swSkStartPt.Z;
    swStartPt = (MathPoint)swMathUtil.CreatePoint(nPt);
    swStartPt = (MathPoint)swStartPt.MultiplyTransform(swSkXform);
    double[] swStartPtArrayData;
    swStartPtArrayData = (double[])swStartPt.ArrayData;

    nPt[0] = swSkEndPt.X;
    nPt[1] = swSkEndPt.Y;
    nPt[2] = swSkEndPt.Z;
    swEndPt = (MathPoint)swMathUtil.CreatePoint(nPt);
    swEndPt = (MathPoint)swEndPt.MultiplyTransform(swSkXform);
    double[] swEndPtArrayData;
    swEndPtArrayData = (double[])swEndPt.ArrayData;

    Debug.Print("File = " + swModel.GetPathName());
    Debug.Print("  Feature = " + swFeat.Name + " [" + swFeat.GetTypeName2() +
"]");
    Debug.Print("    Sketch          = " + swSketchFeat.Name);
    Debug.Print("    SegID          = [" + vID[0] + ", " + vID[1] + "]");
    Debug.Print("    Start with respect to sketch = (" + swSkStartPt.X *
1000.0 + ", " + swSkStartPt.Y * 1000.0 + ", " + swSkStartPt.Z * 1000.0 + ") mm");
    Debug.Print("    End with respect to sketch   = (" + swSkEndPt.X * 1000.0
+ ", " + swSkEndPt.Y * 1000.0 + ", " + swSkEndPt.Z * 1000.0 + ") mm");
    Debug.Print("    Start with respect to model  = (" +
swStartPtArrayData[0] * 1000.0 + ", " + swStartPtArrayData[1] * 1000.0 + ", " +
swStartPtArrayData[2] * 1000.0 + ") mm");
    Debug.Print("    End with respect to model    = (" + swEndPtArrayData[0] *
1000.0 + ", " + swEndPtArrayData[1] * 1000.0 + ", " + swEndPtArrayData[2] * 1000.0 + ")
mm");
    }
}

public SldWorks swApp;
}
}

```

Get Names of Sketch Segments Example (VBA)

This example shows how to get the names of the sketch segments in a sheet metal bend.

```
' -----  
' Preconditions:  
' 1. Open a sheet metal part or a drawing containing a sketched bend.  
' 2. Unsuppress the flat pattern for the bend.  
' 3. Select a bend feature in the FeatureManager design tree.  
'  
' Postconditions: Inspect the Immediate window for information about each  
' sketch segment in the bend feature.  
' -----
```

Option Explicit

```
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim swMathUtil As SldWorks.MathUtility  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swFeat As SldWorks.Feature  
    Dim swOneBend As SldWorks.OneBendFeatureData  
    Dim vSketchSegs As Variant  
    Dim swSketchSeg As SldWorks.SketchSegment  
    Dim swSketch As SldWorks.Sketch  
    Dim swSketchFeat As SldWorks.Feature  
    Dim swSketchLine As SldWorks.SketchLine  
    Dim swSkStartPt As SldWorks.SketchPoint  
    Dim swSkEndPt As SldWorks.SketchPoint  
    Dim swSelData As SldWorks.SelectData  
    Dim nPt(2) As Double  
    Dim vPt As Variant  
    Dim swStartPt As SldWorks.MathPoint  
    Dim swEndPt As SldWorks.MathPoint  
    Dim swSkXform As SldWorks.MathTransform  
    Dim vID As Variant  
    Dim i As Long  
  
    Set swApp = Application.SldWorks  
    Set swMathUtil = swApp.GetMathUtility  
  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
    Set swSelData = swSelMgr.CreateSelectData  
    Set swOneBend = swFeat.GetDefinition  
    Debug.Print "Type of bend (swBendType_e): " & swOneBend.GetType()  
    Debug.Print "Number of sketch segments: " &  
swOneBend.GetFlatPatternSketchSegmentCount2  
    vSketchSegs = swOneBend.FlatPatternSketchSegments2  
  
    For i = 0 To UBound(vSketchSegs)  
        Set swSketchSeg = vSketchSegs(i)  
        Set swSketch = swSketchSeg.GetSketch  
        Set swSketchLine = swSketchSeg  
        Set swSkStartPt = swSketchLine.GetStartPoint2  
        Set swSkEndPt = swSketchLine.GetEndPoint2  
        vID = swSketchSeg.GetID
```

```

' Get sketch feature
Set swSketchFeat = swSketch
Set swSkXform = swSketch.ModelToSketchTransform
Set swSkXform = swSkXform.Inverse

nPt(0) = swSkStartPt.X
nPt(1) = swSkStartPt.Y
nPt(2) = swSkStartPt.Z
vPt = nPt
Set swStartPt = swMathUtil.CreatePoint(vPt)
Set swStartPt = swStartPt.MultiplyTransform(swSkXform)

nPt(0) = swSkEndPt.X
nPt(1) = swSkEndPt.Y
nPt(2) = swSkEndPt.Z
vPt = nPt
Set swEndPt = swMathUtil.CreatePoint(vPt)
Set swEndPt = swEndPt.MultiplyTransform(swSkXform)

Debug.Print "File = " & swModel.GetPathName
Debug.Print "  Feature = " & swFeat.Name & " [" & swFeat.GetType & "]"
Debug.Print "    Sketch = " & swSketchFeat.Name
Debug.Print "    SegID = [" & vID(0) & ", " & vID(1) & "]"
Debug.Print "    Start with respect to sketch = (" & swSkStartPt.X * 1000# & ", " & swSkStartPt.Y * 1000# & ", " & swSkStartPt.Z * 1000# & ") mm"
Debug.Print "    End with respect to sketch = (" & swSkEndPt.X * 1000# & ", " & swSkEndPt.Y * 1000# & ", " & swSkEndPt.Z * 1000# & ") mm"
Debug.Print "    Start with respect to model = (" & swStartPt.ArrayData(0) * 1000# & ", " & swStartPt.ArrayData(1) * 1000# & ", " & swStartPt.ArrayData(2) * 1000# & ") mm"
Debug.Print "    End with respect to model = (" & swEndPt.ArrayData(0) * 1000# & ", " & swEndPt.ArrayData(1) * 1000# & ", " & swEndPt.ArrayData(2) * 1000# & ") mm"
Next i
End Sub

```

Get Names of Sketch Segments Example (VB.NET)

This example shows how to get the names of the sketch segments in a sheet metal bend.

```
' -----  
' Preconditions:  
' 1. Open a sheet metal part or a drawing containing a bend.  
' 2. Unsuppress the flat pattern for the bend.  
' 3. Select a bend feature in the FeatureManager design tree.  
  
' Postconditions: Inspect the Immediate window for information about each  
' sketch segment in the bend feature.  
' -----  
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Sub main()  
  
        Dim swMathUtil As MathUtility  
        Dim swModel As ModelDoc2  
        Dim swSelMgr As SelectionMgr  
        Dim swFeat As Feature  
        Dim swOneBend As OneBendFeatureData  
        Dim vSketchSegs As Object  
        Dim swSketchSeg As SketchSegment  
        Dim swSketch As Sketch  
        Dim swSketchFeat As Feature  
        Dim swSketchLine As SketchLine  
        Dim swSkStartPt As SketchPoint  
        Dim swSkEndPt As SketchPoint  
        Dim swSelData As SelectData  
        Dim nPt(2) As Double  
        Dim vPt As Object  
        Dim swStartPt As MathPoint  
        Dim swEndPt As MathPoint  
        Dim swSkXform As MathTransform  
        Dim vID As Object  
        Dim i As Long  
  
        swMathUtil = swApp.GetMathUtility  
        swModel = swApp.ActiveDoc  
        swSelMgr = swModel.SelectionManager  
        swFeat = swSelMgr.GetSelectedObject6(1, -1)  
        swSelData = swSelMgr.CreateSelectData  
        swOneBend = swFeat.GetDefinition  
        Debug.Print("Type of bend (swBendType_e): " & swOneBend.GetType())  
        Debug.Print("Number of sketch segments: " &  
swOneBend.GetFlatPatternSketchSegmentCount2)  
        vSketchSegs = swOneBend.FlatPatternSketchSegments2  
  
        For i = 0 To UBound(vSketchSegs)  
            swSketchSeg = vSketchSegs(i)  
            swSketch = swSketchSeg.GetSketch  
            swSketchLine = swSketchSeg  
            swSkStartPt = swSketchLine.GetStartPoint2
```

```

swSkEndPt = swSketchLine.GetEndPoint2
vID = swSketchSeg.GetID

' Get sketch feature
swSketchFeat = swSketch
swSkXform = swSketch.ModelToSketchTransform
swSkXform = swSkXform.Inverse

nPt(0) = swSkStartPt.X
nPt(1) = swSkStartPt.Y
nPt(2) = swSkStartPt.Z
vPt = nPt
swStartPt = swMathUtil.CreatePoint(vPt)
swStartPt = swStartPt.MultiplyTransform(swSkXform)

nPt(0) = swSkEndPt.X
nPt(1) = swSkEndPt.Y
nPt(2) = swSkEndPt.Z
vPt = nPt
swEndPt = swMathUtil.CreatePoint(vPt)
swEndPt = swEndPt.MultiplyTransform(swSkXform)

Debug.Print("File = " & swModel.GetPathName)
Debug.Print(" Feature = " & swFeat.Name & " [" & swFeat.GetTypeName2 & "]")
Debug.Print(" Sketch = " & swSketchFeat.Name)
Debug.Print(" SegID = [" & vID(0) & ", " & vID(1) & "]")
Debug.Print(" Start with respect to sketch = (" & swSkStartPt.X * 1000.0#
& ", " & swSkStartPt.Y * 1000.0# & ", " & swSkStartPt.Z * 1000.0# & ") mm")
Debug.Print(" End with respect to sketch = (" & swSkEndPt.X * 1000.0# &
", " & swSkEndPt.Y * 1000.0# & ", " & swSkEndPt.Z * 1000.0# & ") mm")
Debug.Print(" Start with respect to model = (" & swStartPt.ArrayData(0)
* 1000.0# & ", " & swStartPt.ArrayData(1) * 1000.0# & ", " & swStartPt.ArrayData(2) *
1000.0# & ") mm")
Debug.Print(" End with respect to model = (" & swEndPt.ArrayData(0) *
1000.0# & ", " & swEndPt.ArrayData(1) * 1000.0# & ", " & swEndPt.ArrayData(2) * 1000.0# &
") mm")
Next i
End Sub

Public swApp As SldWorks

End Class

```

Get Normal of Planar Face Example (VBA)

This example shows how to get the normal parameters of a planar face.

```
' -----  
' Preconditions:  
' 1. Open a part or assembly.  
' 2. Select a planar face.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the normal vector of the selected planar  
'    face.  
' 2. Examine the Immediate window.  
' -----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swFace As SldWorks.Face2  
    Dim vNorm As Variant  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)  
  
    vNorm = swFace.Normal  
    Debug.Print "Normal = (" & vNorm(0) & ", " & vNorm(1) & ", " & vNorm(2) & ")"  
  
End Sub
```

Get_Normal_of_Planar_Surface_Example_VB (Topic)

SOLIDWORKS API Help

Get Normal of Planar Surface Example (VBA)

This example shows how to the normal parameters of a planar surface.

```
' -----  
' Preconditions:  
' 1. Open a part, assembly, or drawing.  
' 2. Select a face on a planar surface.  
' 3. Open the Immediate window.  
'  
' Postconditions: Examine the Immediate window.  
'  
' NOTE: The normal of a surface (ISurface::PlaneParams)  
' cannot be in the same direction as the normal  
' of the face (IFace2::Normal).  
' -----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swFace As SldWorks.Face2  
    Dim swSurf As SldWorks.Surface  
    Dim vPlaneParam As Variant  
    Dim vFaceNorm As Variant  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swFace = swSelMgr.GetSelectedObject5(1)  
    Set swSurf = swFace.GetSurface: Debug.Assert swSurf.IsPlane  
    vPlaneParam = swSurf.PlaneParams  
    vFaceNorm = swFace.Normal  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print "  Surface"  
    Debug.Print "    normal          = (" & vPlaneParam(0) & ", " & vPlaneParam(1) & ", " &  
vPlaneParam(2) & ")"  
    Debug.Print "    root          = (" & vPlaneParam(3) * 1000# & ", " & vPlaneParam(4) * 1000# &  
", " & vPlaneParam(5) * 1000# & ") mm"  
    Debug.Print "  Face"  
    Debug.Print "    normal          = (" & vFaceNorm(0) & ", " & vFaceNorm(1) & ", " & vFaceNorm(2)  
& ")"  
    Debug.Print "    true if face normal and surface normal in opposite direction, false if in same  
direction = " & swFace.FaceInSurfaceSense  
  
End Sub
```

Get Note By Name Example (VBA)

This example shows how to get a note by its name using two separate methods: note traversal and note selection. The note selection method is faster because it avoids traversing all of the notes on the template.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\assem20.slddrw.  
'  
' Postconditions:  
' 1. Opens the drawing template for editing.  
' 2. Traverses the notes on the drawing template  
'    until the note named DetailItem300 is found.  
' 3. Click OK to close the message box.  
' 4. Selects the note by its name.  
' 5. Changes the text of the note to A.1.  
' 6. Click OK to close the message box.  
' 7. Examine the revision number note in the  
'    drawing's title block.  
'  
' NOTE: Because the drawing is used elsewhere, do not save  
' changes.  
'-----
```

Option Explicit

```
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swDrawing As SldWorks.DrawingDoc  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swNote As SldWorks.note  
    Dim swView As SldWorks.view  
    Dim swSelObj As Object  
    Dim theCurrentRev As String  
    Dim foundNote As Boolean  
    Dim ret As Boolean  
    Dim descriptionText As String  
  
    Const swSelNOTES = 15  
    Const swDocDRAWING = 3  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    ' Verify that the document is a drawing  
    If (swModel Is Nothing) Or (swModel.GetType <> swDocDRAWING) Then  
        Exit Sub  
    End If  
    Set swDrawing = swModel  
  
    ' Edit the current drawing template  
    swDrawing.EditTemplate  
  
    ' This is the template  
    Set swView = swDrawing.GetFirstView  
  
    ' This section shows a slower method for  
    ' selecting a note by name by using note  
    ' traversal (swView::GetFirstNote and SwView::GetNext)  
  
    ' Get the first note object  
    Set swNote = swView.GetFirstNote  
    ' While you have a valid note  
    Do While Not swNote Is Nothing  
        ' Verify if you have the desired note by checking its name  
        ' This can also be done by checking the position or the existing
```

```

' text string in the note
If (swNote.GetName = "DetailItem300") Then
    foundNote = True
    theCurrentRev = swNote.GetText
    swApp.SendMsgToUser ("Note found")
End If
Set swNote = swNote.GetNext
' Continue until no more notes
Loop

' This section shows a faster method
' for selecting a note by using its name
If (foundNote) Then
    ret = swModel.SelectByID("DetailItem300@Sheet Format1", "NOTE", 0, 0, 0)
    'Get the SelectionMgr object
    Set swSelMgr = swModel.SelectionManager
    ' If user has selected something
    If (swSelMgr.GetSelectedObjectCount <> 0) Then
        ' Get the first item in the selection list
        Set swSelObj = swSelMgr.GetSelectedObject2(1)
        ' If selected object is a note
        If (swSelMgr.GetSelectedObjectType(1) = swSelNOTES) Then
            descriptionText = theCurrentRev + "A.1"
            ' Change the text in the note
            ret = swSelObj.SetText(descriptionText)
            ' If change is successful
            If (ret = True) Then
                swApp.SendMsgToUser ("Note text changed")
            ' If name change failed
            Else
                swApp.SendMsgToUser ("Error changing note text")
            End If
        End If
        ' If selection change failed
    Else
        swApp.SendMsgToUser ("Error selecting note")
    End If
End If
End Sub

```

Get Note Text Formatting Properties Example (VBA)

This example shows how to get the text formatting out of a note.

```
' -----  
' Preconditions:  
' 1. Open a part containing a note.  
' 2. Select the note.  
' 3. Open the Immediate window.  
'  
' Postconditions: Examine the Immediate window.  
' -----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim Part As SldWorks.ModelDoc2  
    Dim SelMgr As SldWorks.SelectionMgr  
    Dim Note As SldWorks.Note  
    Dim TextFormat As SldWorks.TextFormat  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set Part = swApp.ActiveDoc  
    If Not Part Is Nothing Then  
        Set SelMgr = Part.SelectionManager()  
        If SelMgr.GetSelectedObjectCount() <> 1 Then  
            MsgBox "Please select a note."  
        ElseIf SelMgr.GetSelectedObjectType(1) <> swSelNOTES Then  
            MsgBox "Please select a note."  
        Else  
            Set Note = SelMgr.GetSelectedObject2(1)  
            Set TextFormat = Note.GetTextFormat  
            Debug.Print "Text format:"  
            Debug.Print "  Bold: " & TextFormat.Bold  
            Debug.Print "  Italic: " & TextFormat.Italic  
            Debug.Print "  Underline: " & TextFormat.Underline  
            Debug.Print "  Strikeout: " & TextFormat.Strikeout  
            Debug.Print "  Typeface: " & TextFormat.TypeFaceName  
            If TextFormat.IsHeightSpecifiedInPts() Then  
                Debug.Print "    Height in points: " & TextFormat.CharHeightInPts  
            End If  
        End If  
    End If  
End Sub
```

Get Notes from New or Existing Title Block (C#)

This example shows how to create a title block in a drawing, if one does not already exist, and how to get the notes from an existing title block in a drawing.

```
//-----  
// Preconditions: Drawing document is open.  
//  
// Postconditions: If the drawing contains a title block, then  
//                  the notes of that block are printed  
//                  to the Immediate window. If not,  
//                  a title block is created.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
using System.Runtime.InteropServices;  
namespace ExampleCS.csproj  
{  
    public partial class SolidWorksMacro  
    {  
        ModelDoc2 swModel;  
        ModelDocExtension swExt;  
        SelectionMgr swSelMgr;  
        View swView;  
        DrawingDoc swDraw;  
        public void Main()  
        {  
            swModel = swApp.ActiveDoc as ModelDoc2;  
            swExt = swModel.Extension;  
            swSelMgr = swModel.SelectionManager as SelectionMgr;  
            swDraw = swModel as DrawingDoc;  
            Sheet swSheet;  
            swSheet = swDraw.GetCurrentSheet() as Sheet;  
            TitleBlock swTitleBlock;  
            swTitleBlock = swSheet.TitleBlock;  
            object[] vNotes;  
            int i;  
            // Create title block if one doesn't exist  
            if (swTitleBlock == null)
```

```

    {
        swView = swDraw.GetFirstView() as View;
        vNotes = (object[])swView.GetNotes();
        // Add first two notes to the title block
        Dispatcher[] notesArray = new Dispatcher[2];
        notesArray[0] = new Dispatcher(vNotes[0]);
        notesArray[1] = new Dispatcher(vNotes[1]);
        swTitleBlock = swSheet.InsertTitleBlock(notesArray);
    }
    vNotes = (object[])swTitleBlock.GetNotes();
    for (i = 0; i < vNotes.Length; i++)
    {
        Note swNote;
        swNote = (Note)vNotes[i];
        Debug.Print("Name: " + swNote.GetName());
        Debug.Print("Value: " + swNote.GetText());
    }
}
/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Get Notes from New or Existing Title Block (VBA)

This example shows how to create a title block in a drawing, if one does not already exist, and how to get the notes from an existing title block in a drawing.

```
' -----  
' Preconditions: Drawing document is open.  
'  
' Postconditions: If the drawing contains a title block, then  
'                  notes of that block are printed  
'                  to the Immediate window. If not, then  
'                  a title block is created.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As ModelDoc2  
Dim swExt As ModelDocExtension  
Dim swSelMgr As SelectionMgr  
Dim swView As View  
Dim swDraw As DrawingDoc  
  
Sub main()  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swExt = swModel.Extension  
Set swSelMgr = swModel.SelectionManager  
  
Set swDraw = swModel  
  
Dim swSheet As Sheet  
Set swSheet = swDraw.GetCurrentSheet  
  
Dim swTitleBlock As TitleBlock  
Set swTitleBlock = swSheet.TitleBlock  
  
Dim vNotes As Variant  
Dim i As Integer  
' Create title block if one doesn't exist  
If swTitleBlock Is Nothing Then
```

```
Set swView = swDraw.GetFirstView
```

```
vNotes = swView.GetNotes
```

```
' Add first two notes to the title block
```

```
Dim notesArray(1) As Object
```

```
Set notesArray(0) = vNotes(0)
```

```
Set notesArray(1) = vNotes(1)
```

```
Set swTitleBlock = swSheet.InsertTitleBlock(notesArray)
```

```
End If
```

```
vNotes = swTitleBlock.GetNotes
```

```
For i = 0 To UBound(vNotes)
```

```
    Dim swNote As Note
```

```
    Set swNote = vNotes(i)
```

```
    Debug.Print "Name: " & swNote.GetName
```

```
    Debug.Print "Value: " & swNote.GetText
```

```
Next
```

```
End Sub
```

Get Notes from New or Existing Title Block (VB.NET)

This example shows how to create a title block in a drawing, if one does not already exist, and how to get the notes from an existing title block in a drawing.

```
' -----  
' Preconditions: Drawing document is open.  
'  
' Postconditions: If the drawing contains a title block, then  
'                  notes of that block are printed  
'                  to the Immediate window. If not, the a  
'                  title block is created.  
' -----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System  
Imports System.Diagnostics  
Imports System.Runtime.InteropServices
```

Partial Class SolidWorksMacro

```
    Dim swModel As ModelDoc2  
    Dim swExt As ModelDocExtension  
    Dim swSelMgr As SelectionMgr  
    Dim swView As View  
    Dim swDraw As DrawingDoc
```

```
    Public Sub main()
```

```
        swModel = swApp.ActiveDoc  
        swExt = swModel.Extension  
        swSelMgr = swModel.SelectionManager
```

```
        swDraw = swModel
```

```
        Dim swSheet As Sheet  
        swSheet = swDraw.GetCurrentSheet
```

```
        Dim swTitleBlock As TitleBlock  
        swTitleBlock = swSheet.TitleBlock
```

```

Dim vNotes As Object
Dim i As Integer
' Create title block if one doesn't exist
If swTitleBlock Is Nothing Then
    swView = swDraw.GetFirstView
    vNotes = swView.GetNotes
    ' Add first two notes to the title block
    Dim notesArray(1) As DispatchWrapper
    notesArray(0) = New DispatchWrapper(vNotes(0))
    notesArray(1) = New DispatchWrapper(vNotes(1))
    swTitleBlock = swSheet.InsertTitleBlock(notesArray)
End If

vNotes = swTitleBlock.GetNotes

For i = 0 To UBound(vNotes)
    Dim swNote As Note
    swNote = vNotes(i)
    Debug.Print("Name: " & swNote.GetName)
    Debug.Print("Value: " & swNote.GetText)
Next

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Get Number of Bodies in Save Bodies Feature (VBA)

This example shows how to get the number of bodies in a Save Bodies feature.

```
' -----  
' Preconditions:  
' 1. Open a part that contains a Save Bodies feature.  
' 2. Select the Save Bodies feature.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the number of bodies in the Save Bodies feature.  
' 2. Examine the Immediate window.  
' -----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swFeat As SldWorks.Feature  
Dim swSaveBodyFeat As SldWorks.SaveBodyFeatureData  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
  
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)  
    Set swSaveBodyFeat = swFeat.GetDefinition  
    swSaveBodyFeat.AccessSelections swModel, Nothing  
    Dim bodyCount As Long  
    bodyCount = swSaveBodyFeat.GetSaveBodiesCount  
    Debug.Print ("Number of bodies in this Save Bodies feature: ") & bodyCount;  
    swSaveBodyFeat.ReleaseSelectionAccess  
  
End Sub
```

Get Number of Instances Skipped in Driving Feature Example (C#)

This example shows how to get the number of instances skipped in the driving feature of a derived pattern feature.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\PatternDrivenComponentPattern.sldasm.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Adds instances to skip in the local linear pattern feature.
// 2. Inserts a derived pattern feature.
// 3. Gets the number of instances skipped in the driving feature for the
//    derived pattern feature.
// 4. Gets whether visual properties are propagated in the derived pattern
//    feature.
// 5. Examine the FeatureManager design tree, graphics area, and
//    Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssemblyDoc = default(AssemblyDoc);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            LocalLinearPatternFeatureData swLocalLinearPatternData = default(LocalLinearPatternFeatureData);

            DerivedPatternFeatureData swDerivedPatternFeatureData = default(DerivedPatternFeatureData);
            int[] list = new int[2];
            object instancesSkipData = null;
            int[] instancesSkipData_Derived = new int[2];
            int instancesToSkipCount = 0;
            bool status = false;
            int i = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Add instances to skip to local linear pattern feature, LocalPattern1
            status = swModelDocExt.SelectByID2("LocalPattern1", "COMPPATTERN", 0, 0, 0, false, 0, null,

0);

            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swLocalLinearPatternData = (LocalLinearPatternFeatureData)swFeature.GetDefinition();
            status = swLocalLinearPatternData.AccessSelections(swModel, null);
            list[0] = 11;
            list[1] = 14;
            instancesSkipData = list;
            swLocalLinearPatternData.SkippedItemArray = instancesSkipData;
            status = swFeature.ModifyDefinition(swLocalLinearPatternData, swModel, null);

            //Insert derived pattern feature, LocalCompPattern1
            status = swModelDocExt.SelectByID2("Part2^patterndrivencomponentpattern-
1@patterndrivencomponentpattern", "COMPONENT", 0, 0, 0, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("LocalPattern1", "COMPPATTERN", 0, 0, 0, true, 6, null, 0

);

            swAssemblyDoc = (AssemblyDoc)swModel;
            status = swAssemblyDoc.InsertDerivedPattern();
        }
    }
}
```

```

        //Get number of instances skipped in the driving feature, LocalPattern1,
        //for derived pattern feature, LocalCompPattern1
        status = swModelDocExt.SelectByID2("LocalCompPattern1", "COMPPATTERN", 0, 0, 0, false, 0, nul
1, 0);

        swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
        swDerivedPatternFeatureData = (DerivedPatternFeatureData)swFeature.GetDefinition();
        status = swDerivedPatternFeatureData.AccessSelections(swModel, null);
        instancesSkipData_Derived = (int[])swDerivedPatternFeatureData.DrivingFeatureSkippedItemArray
;
        instancesToSkipCount = instancesSkipData_Derived.Length;
        Debug.Print("Number of instances skipped in the driving feature of the derived pattern
feature: " + instancesToSkipCount);
        for (i = 0; i <= instancesToSkipCount - 1; i++)
        {
            Debug.Print(" Instance skipped: " + instancesSkipData_Derived[i]);
        }

        //Get whether visual properties are propagated in the derived pattern feature
        Debug.Print ("Visual properties propagated in the derived pattern feature? " +
swDerivedPatternFeatureData.PropagateVisualProperty);

        swDerivedPatternFeatureData.ReleaseSelectionAccess();

    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}

```

Get Number of Instances Skipped in Driving Feature Example (VBA)

This example shows how to get the number of instances skipped in the driving feature of a derived pattern feature.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\PatternDrivenComponentPattern.sldasm.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Adds instances to skip in the local linear pattern feature.  
' 2. Inserts a derived pattern feature.  
' 3. Gets the number of instances skipped in the driving feature for the  
'    derived pattern feature.  
' 4. Gets whether visual properties are propagated in the derived pattern  
'    feature.  
' 5. Examine the FeatureManager design tree, graphics area, and  
'    Immediate window.  
'  
' NOTE: Because the assembly is used elsewhere, do not save changes.  
' -----
```

Option Explicit

```
Dim SwApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swAssemblyDoc As SldWorks.AssemblyDoc  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSelectionMgr As SldWorks.SelectionMgr  
Dim swFeature As SldWorks.Feature  
Dim swLocalLinearPatternData As SldWorks.LocalLinearPatternFeatureData  
Dim swDerivedPatternFeatureData As SldWorks.DerivedPatternFeatureData  
Dim list(1) As Long  
Dim instancesSkipData As Variant  
Dim instancesSkipData_Derived As Variant  
Dim instancesToSkipCount As Long  
Dim status As Boolean  
Dim i As Long
```

Sub main()

```
Set SwApp = Application.SldWorks  
Set swModel = SwApp.ActiveDoc  
Set swModelDocExt = swModel.Extension  
  
'Add instances to skip to local linear pattern feature, LocalPattern1  
status = swModelDocExt.SelectByID2("LocalPattern1", "COMPPATTERN", 0, 0, 0, False, 0, Nothing, 0)  
Set swSelectionMgr = swModel.SelectionManager  
Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)  
Set swLocalLinearPatternData = swFeature.GetDefinition  
status = swLocalLinearPatternData.AccessSelections(swModel, Nothing)  
list(0) = 11  
list(1) = 14  
instancesSkipData = list  
swLocalLinearPatternData.SkippedItemArray = instancesSkipData  
status = swFeature.ModifyDefinition(swLocalLinearPatternData, swModel, Nothing)  
  
'Insert derived pattern feature, LocalCompPattern1  
status = swModelDocExt.SelectByID2("Part2^patterndrivencomponentpattern-  
1@patterndrivencomponentpattern", "COMPONENT", 0, 0, 0, False, 1, Nothing, 0)  
status = swModelDocExt.SelectByID2("LocalPattern1", "COMPPATTERN", 0, 0, 0, True, 6, Nothing, 0)  
Set swAssemblyDoc = swModel  
status = swAssemblyDoc.InsertDerivedPattern()
```

```

'Get number of instances skipped in the driving feature, LocalPattern1,
'for the derived pattern feature, LocalCompPattern1
status = swModelDocExt.SelectByID2("LocalCompPattern1", "COMPPATTERN", 0, 0, 0, False, 0, Nothing, 0)
Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
Set swDerivedPatternFeatureData = swFeature.GetDefinition
status = swDerivedPatternFeatureData.AccessSelections(swModel, Nothing)
instancesSkipData_Derived = swDerivedPatternFeatureData.DrivingFeatureSkippedItemArray
instancesToSkipCount = UBound(instancesSkipData_Derived)
Debug.Print "Number of instances skipped in the driving feature of the derived pattern feature: " &
instancesToSkipCount + 1
For i = 0 To instancesToSkipCount
    Debug.Print "    Instance skipped: " & instancesSkipData_Derived(i)
Next i
swDerivedPatternFeatureData.ReleaseSelectionAccess

'Get whether visual properties are propagated in the derived pattern feature
Debug.Print "Visual properties propagated in the derived pattern feature? " &
swDerivedPatternFeatureData.PropagateVisualProperty

End Sub

```

Get Number of Instances Skipped in Driving Feature Example (VB.NET)

This example shows how to get the number of instances skipped in the driving feature of a derived pattern feature.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\PatternDrivenComponentPattern.sldasm.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Adds instances to skip in the local linear pattern feature.
' 2. Inserts a derived pattern feature.
' 3. Gets the number of instances skipped in the driving feature for the
'    derived pattern feature.
' 4. Gets whether visual properties are propagated in the derived pattern
'    feature.
' 5. Examine the FeatureManager design tree, graphics area, and
'    Immediate window.
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swAssemblyDoc As AssemblyDoc
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swLocalLinearPatternData As LocalLinearPatternFeatureData
        Dim swDerivedPatternFeatureData As DerivedPatternFeatureData
        Dim list(1) As Integer
        Dim instancesSkipData As Object
        Dim instancesSkipData_Derived(1) As Integer
        Dim instancesToSkipCount As Integer
        Dim status As Boolean
        Dim i As Integer

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        'Add instances to skip to local linear pattern feature, LocalPattern1
        status = swModelDocExt.SelectByID2("LocalPattern1", "COMPPATTERN", 0, 0, 0, False, 0, Nothing, 0)

    )

        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swLocalLinearPatternData = swFeature.GetDefinition
        status = swLocalLinearPatternData.AccessSelections(swModel, Nothing)
        list(0) = 11
        list(1) = 14
        instancesSkipData = list
        swLocalLinearPatternData.SkippedItemArray = instancesSkipData
        status = swFeature.ModifyDefinition(swLocalLinearPatternData, swModel, Nothing)

        'Insert derived pattern feature, LocalCompPattern1
        status = swModelDocExt.SelectByID2("Part2^patterndrivencomponentpattern-
1@patterndrivencomponentpattern", "COMPONENT", 0, 0, 0, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("LocalPattern1", "COMPPATTERN", 0, 0, 0, True, 6, Nothing, 0)
        swAssemblyDoc = swModel
        status = swAssemblyDoc.InsertDerivedPattern()

        'Get number of instances skipped in the driving feature, LocalPattern1,
        'for the derived pattern feature, LocalCompPattern1
        status = swModelDocExt.SelectByID2("LocalCompPattern1", "COMPPATTERN", 0, 0, 0, False, 0, Nothing
, 0)

        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    
```

```

swDerivedPatternFeatureData = swFeature.GetDefinition
status = swDerivedPatternFeatureData.AccessSelections(swModel, Nothing)
instancesSkipData_Derived = swDerivedPatternFeatureData.DrivingFeatureSkippedItemArray
instancesToSkipCount = UBound(instancesSkipData_Derived)
Debug.Print("Number of instances skipped in the driving feature of the derived pattern
feature: " & instancesToSkipCount + 1)
For i = 0 To instancesToSkipCount
    Debug.Print(" Instance skipped: " & instancesSkipData_Derived(i))
Next i

'Get whether visual properties are propagated in the derived pattern feature
Debug.Print("Visual properties propagated in the derived pattern feature? " &
swDerivedPatternFeatureData.PropagateVisualProperty)

swDerivedPatternFeatureData.ReleaseSelectionAccess()

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get_Number_of_Lines_Flat-pattern_Drawing_View_Boundary-box_Sketch_Example_CSharp (Topic)

SOLIDWORKS API Help

Get the Number of Lines in Flat-pattern Drawing View's Boundary-box Sketch (C#)

This example shows how to get the number of lines in a flat-pattern drawing view's boundary-box sketch.

```
//-----  
// Preconditions:  
// 1. Open public_documents\samples\tutorial\api\SMGussetAPI.SLDPRT.  
// 2. Create a new drawing document.  
// 3. Select SMGussetAPI.SLDPRT in the View  
//    Palette's dropdown list box.  
// 4. Open the Immediate window.  
//  
// Postconditions:  
// 1. Examine the Immediate window and the drawing.  
// 2. If necessary, drag the drawing onto the drawing sheet  
//    and zoom in on the drawing view.  
//  
// NOTE: Because the part is used elsewhere, do not save  
// changes.  
//-----
```

```
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System;  
using System.Diagnostics;  
  
namespace GetSMBoundingBoxDisplayDataViewCSharp.csproj  
{  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
  
            ModelDoc2 swModel;  
            DrawingDoc swDrawing;  
            View swView;  
            Sheet swSheet;  
            DisplayData swDisplayData;  
            double[] sheetProperties = null;  
            double sheetScale = 0;  
            swDwgPaperSizes_e paperSize;  
            double width = 0;  
            double height = 0;  
            long numViews = 0;  
            object[] viewNames = null;  
            string viewPaletteName = "";  
            string drawingViewName = "";  
            int i = 0;  
            bool status = false;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swDrawing = (DrawingDoc)swModel;  
  
            // Get current sheet  
            swSheet = (Sheet)swDrawing.GetCurrentSheet();  
            sheetProperties = (double[])swSheet.GetProperties();
```

```

sheetScale = (double)sheetProperties[2] / sheetProperties[3];
paperSize = (swDwgPaperSizes_e)swSheet.GetSize(ref width, ref height);

// Get number of views on View Palette
numViews = 0;
viewNames = (object[])swDrawing.GetDrawingPaletteViewNames();

// Iterate through views on View Palette
// When view name equals "Flat pattern", drop
// that view in drawing
if (!((viewNames == null)))
{
    numViews = viewNames.GetUpperBound(0) - viewNames.GetLowerBound(0);
    for (i = 0; i <= numViews; i++)
    {
        viewPaletteName = (string)viewNames[i];
        if ((viewPaletteName == "Flat pattern"))
        {
            Debug.Print("Dropping View Palette view named: " + viewPaletteName);
            swView =
(View)swDrawing.DropDrawingViewFromPalette2(viewPaletteName, 0.0, 0.0, 0.0);
            drawingViewName = swView.GetName2();
            Debug.Print("Dropped View Palette view into drawing view named: " +
drawingViewName);
            break;
        }
    }
}

// Activate view and get number of lines in
// its boundary box sketch
status = swDrawing.ActivateView(drawingViewName);
swView = (View)swDrawing.ActiveDrawingView;

// Make sure the bounding box sketch is visible
status = ((ModelDocExtension)swModel.Extension).SelectByID2("Bounding-
Box2@SMGussetAPI-1@DrawingView1", "SKETCH", 0, 0, 0, false, 0, null, 0);
swModel.UnblankSketch();
swDisplayData = (DisplayData)swView.GetSMBoundaryBoxDisplayData2();
Debug.Print("Number of lines in boundary box of flat-pattern drawing view: " +
swDisplayData.GetLineCount());
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;

}

}

```

Get_Number_of_Lines_Flat-pattern_Drawing_View_Boundary-box_Sketch_Example_VB

(Topic)

SOLIDWORKS API Help

Get the Number of Lines in Flat-pattern Drawing View's Boundary-box Sketch (VBA)

This example shows how to get the number of lines in a flat-pattern drawing view's boundary-box sketch.

```
' -----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\SMGussetAPI.SLDPRT.  
' 2. Create a new drawing document.  
' 3. Select SMGussetAPI.SLDPRT in the View  
'    Palette's dropdown list box.  
' 4. Open the Immediate window.  
'  
' Postconditions:  
' 1. Examine the Immediate window and the drawing.  
' 2. If necessary, drag the drawing onto the drawing sheet  
'    and zoom in on the drawing view.  
'  
' NOTE: Because the part is used elsewhere, do not save  
' changes.  
' -----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swDrawing As SldWorks.DrawingDoc  
Dim swView As SldWorks.View  
Dim swSheet As SldWorks.Sheet  
Dim swDisplayData As SldWorks.DisplayData  
Dim sheetProperties As Variant  
Dim sheetScale As Double  
Dim paperSize As swDwgPaperSizes_e  
Dim width As Double  
Dim height As Double  
Dim numViews As Long  
Dim viewName As Variant  
Dim viewNames As Variant  
Dim viewPaletteName As String  
Dim drawingViewName As String  
Dim status As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swDrawing = swModel  
  
    ' Get current sheet  
    Set swSheet = swDrawing.GetCurrentSheet  
    sheetProperties = swSheet.GetProperties  
    sheetScale = sheetProperties(2) / sheetProperties(3)  
    paperSize = swSheet.GetSize(width, height)  
  
    ' Get number of views on View Palette  
    numViews = 0  
    viewNames = swDrawing.GetDrawingPaletteViewNames  
  
    ' Iterate through views on View Palette  
    ' When view name equals "Flat pattern", drop  
    ' that view in drawing  
    If (Not (IsEmpty(viewNames))) Then  
        numViews = (UBound(viewNames) - LBound(viewNames) + 1)
```

```

        For Each viewName In viewNames
            viewPaletteName = viewName
            If (viewPaletteName = "Flat pattern") Then
                Debug.Print "Dropping View Palette view named: " & viewPaletteName
                Set swView = swDrawing.DropDrawingViewFromPalette2(viewPaletteName, 0#, 0#,
0#)

                drawingViewName = swView.GetName2
                Debug.Print "Dropped View Palette view into drawing view named: " &
drawingViewName
            Exit For
            End If
        Next viewName
    End If

    ' Activate view and get number of lines in
    ' its boundary box sketch
    status = swDrawing.ActivateView(drawingViewName)
    Set swView = swDrawing.ActiveDrawingView

    ' Make sure the bounding box sketch is visible
    status = swModel.Extension.SelectByID2("Bounding-Box2@SMGussetAPI-1@DrawingView1",
"SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    swModel.UnblankSketch

    Set swDisplayData = swView.GetSMBoundaryBoxDisplayData2
    Debug.Print "Number of lines in boundary box of flat-pattern drawing view: " &
swDisplayData.GetLineCount

End Sub

```